

Hosty — Informe Final (versión consolidada)

Documento consolidado — generado a partir del vault `documentacion-final/`, fecha: 2026-07-29.

Este archivo reúne, en un único documento portable, las 21 notas de contenido del informe final de Hosty (16 secciones numeradas + 5 anexos), en su orden de lectura canónico. Es una concatenación sin pérdida de esas notas: no reemplaza al vault de Obsidian. Las fuentes editables e individuales de cada sección — y las notas de apoyo (`_meta/`) que este documento no incluye — siguen viviendo en `documentacion-final/` ; cualquier corrección de contenido debe hacerse ahí y volver a generar este archivo.

Tabla de contenidos

- [00. Portada y Ficha Técnica](#)
- [01. Resumen Ejecutivo](#)
- [02. Acrónimos](#)
- [03. Introducción](#)
- [04. Objetivos](#)
- [05. Problema a Resolver](#)
- [06. Impacto de la Solución](#)
- [07. Equipo y Roles](#)
- [08. Diseño y Desarrollo](#)
- [09. Planificación Scrum](#)
- [10. Presupuesto](#)
- [11. Arquitectura](#)
- [12. Testing y Calidad](#)
- [13. Ejecución por Sprint](#)
- [14. Métricas](#)
- [15. Conclusiones](#)
- [Anexo I. Modelo de Datos](#)
- [Anexo II. Diagramas de Flujo Complementarios](#)
- [Anexo III. Backlog Completo de User Stories](#)
- [Anexo IV. API y Repositorio](#)
- [Anexo V. Evidencias de QA](#)

00. Portada y Ficha Técnica

Portada

Hosty es una plataforma web de tipo marketplace para la búsqueda, comparación y reserva de salones de eventos en la provincia de Tucumán, Argentina.

✓ **PLACEHOLDER P-01 — Eslogan del proyecto** Completar con el eslogan o la bajada conceptual definitiva de Hosty antes de la entrega. Responsable: equipo. Destino: portada.

✓ **PLACEHOLDER P-02 — Materia** Completar con el nombre de la materia o asignatura en la que se presenta este informe. Responsable: equipo. Destino: portada.

✓ **PLACEHOLDER P-03 — Carrera** Completar con el nombre de la carrera. Responsable: equipo. Destino: portada y pie de página del documento exportado (ver instrucción de exportación a PDF, más abajo).

✓ **PLACEHOLDER P-04 — Institución** Completar con la denominación formal de la institución educativa. Responsable: equipo. Destino: portada.

✓ **PLACEHOLDER P-05 — Año** Completar con el año de cursada o de presentación del informe. Responsable: equipo. Destino: portada.

Ficha técnica del proyecto

Campo	Valor
Nombre del proyecto	Hosty
Eslogan	(ver P-01)
Materia	(ver P-02)
Carrera	(ver P-03)
Institución	(ver P-04)
Año	(ver P-05)
Integrantes y roles formales	(ver P-06)
Metodología	Scrum, con iteraciones (sprints)
Período de desarrollo	2026-03-29 – 2026-06-24 (sprints S1–S5)
Repositorio	https://github.com/juanpabloaldez/hosty

Campo	Valor
URLs de producción	(ver P-07)
Versión de este documento	v0.1 (borrador)

Tabla 1 — Ficha técnica del proyecto.

✓ **PLACEHOLDER P-06 — Nombres, legajos y roles formales del equipo** Completar con los nombres reales, legajos y rol formal (no de equipo Scrum) de los cinco integrantes. La distribución de identidades Git y de commits por persona ya está verificada en [_\(ver documentacion-final/meta/Datos-Verificables.md\)](#) (M05); sólo falta la denominación formal. Responsable: equipo. Destino: Tabla 1 y la tabla de integrantes de [Equipo y Roles](#).

✓ **PLACEHOLDER P-07 — URLs de producción** Completar con las URLs públicas de despliegue (frontend y, si corresponde, panel de Supabase) una vez confirmadas por el equipo. Responsable: equipo. Destino: Tabla 1.

Índice numerado

0. Portada y Ficha Técnica
1. Resumen Ejecutivo
2. Acrónimos
3. Introducción
4. Objetivos
5. Problema a Resolver
6. Impacto de la Solución
7. Equipo y Roles
8. Diseño y Desarrollo
9. Planificación Scrum
10. Presupuesto
11. Arquitectura
12. Testing y Calidad
13. Ejecución por Sprint
14. Métricas
15. Conclusiones Anexo I. Modelo de Datos Anexo II. Diagramas de Flujo Complementarios Anexo III. Backlog de User Stories Anexo IV. API y Repositorio Anexo V. Evidencias de QA

El detalle navegable de este índice, con enlaces a cada nota, se encuentra en [_\(ver documentacion-final/meta/Indice.md\)](#).

Control de versiones del documento

Versión	Fecha	Cambios	Responsable
v0.1	2026-07-28	Generación inicial del vault <code>documentacion-final/</code> (Lote 0 — Fundación)	Agente SDD

Tabla 2 — Control de versiones del documento.

✓ **PLACEHOLDER P-08** — Versión final del documento y fecha de defensa Completar la versión definitiva del informe y la fecha de defensa una vez cerrado el proceso de revisión. Responsable: equipo. Destino: Tabla 2.

Nota metodológica sobre el origen de la información

Nota metodológica sobre el origen de la información

Este informe distingue de manera explícita tres tipos de contenido. **(a) Datos verificados:** extraídos del historial Git del repositorio, de la API de GitHub y de los archivos de migración del proyecto; su origen se cita en un bloque `[!info] Fuente` que incluye el identificador de la métrica y el comando que permite reproducirla, y se consolidan en la nota `_(ver documentacion-final/meta/Datos-Verificables.md)`. **(b) Contenido simulado:** redactado de forma plausible por no existir registro documental del hecho (retrospectivas, entrevistas, estimaciones de esfuerzo y presupuesto); se señala con `[!warning] Dato simulado` e indica la base sobre la que se reconstruyó. Ningún contenido simulado debe interpretarse como evidencia empírica. **(c) Contenido pendiente:** información que únicamente el equipo puede aportar (denominación institucional, nombres y roles formales, tarifas, capturas de pantalla y URLs productivas); se señala con `[!todo] PLACEHOLDER` y se consolida en `_(ver documentacion-final/meta/Pendientes.md)`.

Instrucción de exportación a PDF

Markdown no admite encabezados ni pies de página. Al exportar el vault a PDF desde Obsidian (*Archivo* → *Exportar a PDF*), debe configurarse el pie de página del documento exportado con: nombre del proyecto (Hosty), carrera, y número de página. Ninguna nota de este vault renderiza un pie de página por sí misma.

01. Resumen Ejecutivo

En la provincia de Tucumán, la búsqueda, comparación y reserva de un salón de eventos depende todavía de canales informales y dispersos: recomendaciones personales, publicaciones en redes sociales o llamados telefónicos, sin un canal único que permita comparar disponibilidad, precio y condiciones entre distintas opciones (ver [Introducción](#) para el desarrollo completo de este contexto).

Hosty es una plataforma web de tipo *marketplace* que centraliza la búsqueda, la comparación y la reserva de salones de eventos, vinculando directamente a dos tipos de usuario: el organizador, que necesita encontrar y reservar un salón acorde a su presupuesto y ubicación, y el propietario o anfitrión, que necesita mayor visibilidad comercial y una gestión centralizada de las reservas que recibe.

La propuesta de valor de Hosty se apoya en tres capacidades centrales: un catálogo público con búsqueda, filtros y visualización geolocalizada de salones; un flujo de reserva guiado que valida disponibilidad y horarios antes de confirmar; y un panel de gestión para el anfitrión, con calendario de reservas y cotización de precio por evento. El alcance del MVP entregado cubre estas tres capacidades de punta a punta, junto con un sistema de favoritos para el organizador y un plan de suscripción "Destacado" que mejora la posición del salón dentro del catálogo. Dos capacidades quedaron fuera del alcance entregado y se documentan explícitamente como diferidas: la integración de cobro con Mercado Pago, tanto para las reservas como para el plan Destacado, y un sistema de reseñas y calificaciones de usuarios.

El proyecto se desarrolló bajo la metodología Scrum, en iteraciones (sprints) sucesivas, a lo largo de un período de aproximadamente 12,6 semanas, entre el 2026-03-29 y el 2026-06-24. Desde el punto de vista técnico, Hosty es una aplicación de una sola página (SPA) construida en React 19, sin servidor de aplicación propio: la persistencia, la autenticación y la autorización se apoyan íntegramente en Supabase como plataforma de *backend as a service* (BaaS), con control de acceso a los datos basado en la propiedad de cada registro y no en un esquema de roles.

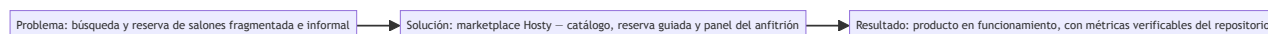


Figura 1 — Síntesis: problema → solución (marketplace de salones) → resultados verificables.

Cifra	Valor	Fuente
Duración del proyecto	~12,6 semanas (2026-03-29 – 2026-06-24)	M03, M04
Commits en <code>dev</code>	181	M01
Contribuidores	5 (9 identidades Git)	M05
Issues cerradas / totales	45 / 50	M06
Pull requests mergeados / totales	26 / 48	M07
Épicas / milestones	7	M08
Rutas totales / protegidas	14 / 8	M09
Tablas del modelo de datos	6	M10
Pruebas automatizadas / archivos de prueba	66 / 18	M12, M13

Tabla 3 — Cifras clave del proyecto.

¡ Fuente — Todos los valores de esta tabla se citan textualmente desde la nota Datos-Verificables (M01, M03, M04, M05, M06, M07, M08, M09, M10, M12, M13); no se recalculan ni se aproximan en esta nota.

El detalle de estas métricas y su interpretación se desarrolla en [Métricas](#); el balance final entre lo planificado y lo entregado se documenta en [Conclusiones](#).

02. Acrónimos

Esta sección reúne, en orden alfabético, las siglas y los términos técnicos utilizados a lo largo del informe. Para cada uno se indica su significado y, cuando corresponde, una aclaración sobre su aplicación concreta en la arquitectura de Hosty — en particular en los casos en que el proyecto se aparta de la implementación más habitual del término, como ocurre con JWT, RBAC y ORM/ODM. El resto de los acrónimos se define de forma estándar, sin adaptaciones particulares al proyecto. Cuando corresponde, cada fila remite al número de la sección del informe donde el concepto se desarrolla con mayor detalle.

Sigla	Significado	Aplicación en Hosty
API	Application Programming Interface	Hosty consume la API REST auto-generada por PostgREST (Supabase); no expone una API propia.
BaaS	Backend as a Service	Supabase actúa como BaaS: no existe un servidor de aplicación propio en el proyecto.
CI/CD	Integración continua / despliegue continuo	GitHub Actions (<code>frontend-tests.yml</code> , <code>web-dev.yml</code> , <code>infra-ci.yml</code>).
CRUD	Create, Read, Update, Delete	Operaciones básicas sobre las 6 tablas del esquema <code>public</code> .
DER	Diagrama de Entidad-Relación	Modelo de datos completo, documentado en el Anexo I.
E2E	End to End	Pruebas automatizadas con Playwright sobre flujos completos de usuario.
JWT	JSON Web Token	Supabase Auth emite internamente un JWT por sesión; Hosty no implementa un servicio de JWT propio ni maneja tokens manualmente, sino que delega la autenticación completa en las sesiones de Supabase Auth.
MVP	Producto Mínimo Viable	Alcance funcional entregado en este proyecto (ver sección 01).
ORM/ODM	Object-Relational / Object-Document Mapping	Hosty no utiliza un ORM: accede a los datos mediante <code>supabase-js</code> sobre la API PostgREST y tipos TypeScript generados por introspección del esquema.
PR	Pull Request	Unidad de integración de código en GitHub.
QA	Quality Assurance	Aseguramiento de calidad, cubierto por pruebas automatizadas y manuales (ver sección 12).
RBAC	Role-Based Access Control	Hosty no implementa RBAC: la autorización es por propiedad (<i>ownership</i>) vía RLS (ver sección 07).
RLS	Row Level Security	Mecanismo de Postgres que restringe las filas visibles o editables según <code>auth.uid()</code> .
SPA	Single Page Application	Arquitectura del frontend, construido en React 19.

Sigla	Significado	Aplicación en Hosty
SQL	Structured Query Language	Lenguaje de consulta de la base de datos Postgres.
UX/UI	Experiencia de usuario / Interfaz de usuario	Diseño funcional y visual del producto (ver sección 08).

Tabla 4 — Glosario de acrónimos y términos.

03. Introducción

Contexto y dominio

El mercado de salones de eventos en la provincia de Tucumán está fragmentado: la oferta de espacios para fiestas, casamientos, cumpleaños y eventos corporativos no se concentra en ningún canal digital especializado. Su descubrimiento depende, en la práctica, de recomendaciones personales, publicaciones en redes sociales de alcance limitado o directorios genéricos que no están orientados a este rubro y que no permiten comparar disponibilidad, precio ni condiciones entre distintas opciones. Como consecuencia, un organizador de eventos no cuenta con una forma sistemática de evaluar alternativas antes de comprometerse con un salón, y un propietario que recién comienza a ofrecer su espacio no dispone de un canal propio para ganar visibilidad frente a organizadores que todavía no lo conocen. La coordinación de la reserva en sí —fecha, horario, condiciones del evento— también queda librada a intercambios informales por mensajería o llamada telefónica, sin un registro estructurado que ambas partes puedan consultar.

Hosty surge como respuesta directa a esta fragmentación: propone un punto de encuentro digital único entre organizadores y propietarios, que reemplaza la búsqueda dispersa por un catálogo consultable, y la coordinación manual de la reserva por un flujo guiado con validación de disponibilidad. El desarrollo del producto tomó como referencia el comportamiento observable de mercados similares (alojamiento, servicios para eventos) para definir cuáles de estas capacidades constituían el núcleo mínimo indispensable del producto.

Relevamiento de requerimientos

El alcance funcional del producto entregado se definió a partir del documento `Definicion de MVP – HOSTY-2026040419562816.pdf`, elaborado por el equipo al inicio del proyecto como especificación de referencia del producto a construir. Este documento fue el que fijó, en última instancia, cuáles funcionalidades formaban parte del MVP (catálogo, reserva, panel del anfitrión) y cuáles quedaban fuera de su alcance inicial, como el cobro en línea o las reseñas de usuarios (ver sección 01).

⚠ Dato simulado SIM-01 — Metodología previa de relevamiento Este cambio de documentación no tuvo acceso a actas de entrevistas, encuestas u otro registro documental adicional que respalde el proceso concreto de relevamiento previo a la redacción del documento de alcance del MVP. Cualquier afirmación sobre el método específico utilizado (entrevistas a organizadores de eventos, encuestas a propietarios de salones, relevamiento de la competencia) que no esté contenida verbatim en dicho documento debe interpretarse como una reconstrucción plausible y no como un registro verificado.

Alcance de este documento

Este informe documenta el producto, su arquitectura técnica, el proceso de gestión bajo el que se construyó y la evidencia de calidad reunida durante el desarrollo. Deliberadamente, no reproduce un manual de usuario final exhaustivo ni transcribe el código fuente completo del proyecto: ambos

elementos ya están disponibles en el repositorio y su transcripción no aportaría valor adicional a una audiencia académica. La siguiente tabla resume, de forma explícita, qué contenidos entran y cuáles quedan fuera del alcance de este documento.

Incluye	No incluye
Descripción funcional del producto y su arquitectura técnica	Manual de usuario final extenso, paso a paso, por cada pantalla
Proceso de gestión del proyecto (Scrum, épicas, sprints, backlog)	Código fuente completo (se referencia el repositorio, no se transcribe)
Estrategia y evidencias de calidad (testing, incidencias)	Actas originales de entrevistas o encuestas no documentadas en el repositorio
Métricas verificables del repositorio y del proceso, con su fuente citada	Capturas de pantalla ya incorporadas — quedan como pendiente en el Anexo V

Tabla 5 — Alcance incluido y excluido.

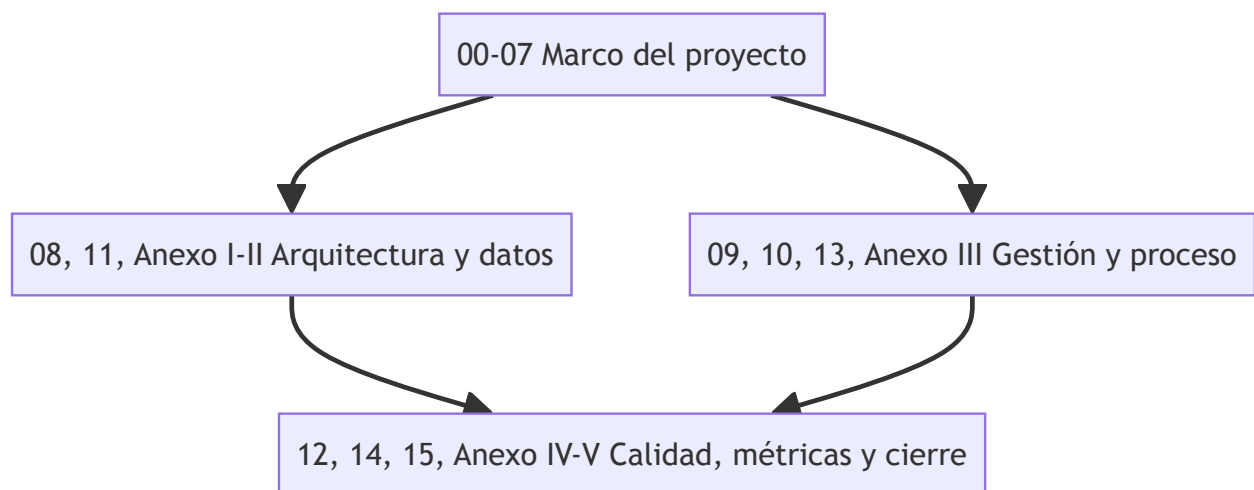


Figura 2 — Estructura del informe: 16 secciones + 5 anexos y sus dependencias de lectura.

Los objetivos que se desprenden de este contexto se desarrollan en [Objetivos](#), y el problema central junto con sus consecuencias se detalla en [Problema a Resolver](#).

04. Objetivos

Objetivo general

OG — Desarrollar y poner en funcionamiento una plataforma web que centralice la búsqueda, la comparación y la reserva de salones de eventos en la provincia de Tucumán, vinculando a organizadores con propietarios. Este objetivo general resume el propósito completo del proyecto y se traduce operativamente en los seis objetivos específicos que se detallan a continuación, cada uno acotado a una capacidad concreta del producto y verificable contra el estado actual del repositorio.

Objetivos específicos

A partir del objetivo general se derivan seis objetivos específicos (OE1–OE6). Cada uno delimita una capacidad concreta del sistema y se acompaña de un criterio de verificación que permite confirmar, contra el estado real del repositorio, si la capacidad fue efectivamente entregada.

Objetivo	Descripción	Criterio de verificación
OE1	Implementar un catálogo público de salones con búsqueda, filtros y visualización geolocalizada	Ruta <code>/salones</code> operativa con filtros y mapa (Leaflet + geocodificación Nominatim)
OE2	Proveer autenticación de usuarios y control de acceso a los datos basado en propiedad	Sesiones de Supabase Auth + guardas <code>requireAuth</code> sobre 8 de las 14 rutas del frontend (M09)
OE3	Habilitar un flujo de reserva guiado con validación de disponibilidad y de horarios	Wizard de reserva de 3 pasos (M20), con verificación de bloqueos de disponibilidad
OE4	Ofrecer al propietario un panel de gestión de sus salones y de las reservas recibidas	Panel del anfitrión con calendario y cotización de precio por reserva
OE5	Asegurar la calidad mediante pruebas automatizadas e integración continua	66 pruebas automatizadas (M12) y 3 workflows de CI/CD (M14)
OE6	Documentar la arquitectura, el proceso y las métricas del proyecto de forma trazable	Este mismo vault: 28 notas con toda métrica citada a su fuente en la nota Datos-Verificables

Tabla 6 — Objetivos específicos y criterio de verificación.

Objetivo de calidad

El objetivo de calidad definido para el proyecto consiste en sostener una suite de pruebas automatizadas que cubra los flujos críticos del frontend. A la fecha de verificación de este informe existen 66 pruebas automatizadas distribuidas en 18 archivos de prueba — 13 pruebas unitarias y de componente con Vitest y Testing Library, más 5 especificaciones end-to-end con Playwright — (M12, M13). El proyecto no tiene configurada una herramienta de cobertura de código (por ejemplo, un reporte de `@vitest/coverage-v8`), por lo que este informe no reporta ni infiere un porcentaje de cobertura: hacerlo sin una fuente verificable contradiría el principio de trazabilidad de métricas que rige todo este documento (ver sección 12, Testing y Calidad).

Trazabilidad objetivo → épica → funcionalidad

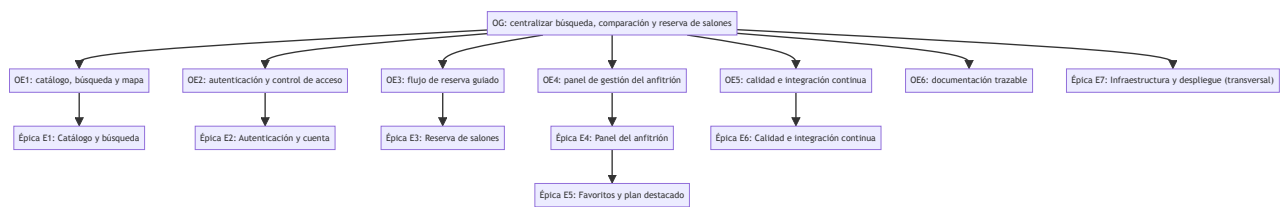


Figura 3 — Árbol de objetivos: OG → OE1..OE6, con la épica asociada a cada OE.

La siguiente tabla conecta cada objetivo específico con la épica temática correspondiente, la funcionalidad concreta entregada y la evidencia verificable que respalda la afirmación de que dicha funcionalidad efectivamente existe en el producto.

Objetivo	Épica asociada	Funcionalidad entregada	Evidencia
OE1	E1 — Catálogo y búsqueda	Catálogo público con filtros y mapa	M09, M15
OE2	E2 — Autenticación y cuenta	Sesiones de Supabase Auth, guarda <code>requireAuth</code> , RLS por <code>auth.uid()</code>	M09
OE3	E3 — Reserva de salones	Wizard de reserva de 3 pasos; estados <code>pending</code> / <code>confirmed</code> / <code>declined</code> / <code>cancelled</code>	M17, M20
OE4	E4 — Panel del anfitrión; E5 — Favoritos y plan destacado	Panel de calendario y cotización; favoritos; plan Destacado (cobro con Mercado Pago diferido, issue #45 abierto)	M10
OE5	E6 — Calidad e integración continua	66 pruebas automatizadas y 3 workflows de CI/CD	M12, M13, M14
OE6	E7 — Infraestructura y despliegue transversal	Documentación trazable del proyecto (este vault) y despliegue automatizado vía GitHub Actions	M14

Tabla 7 — Trazabilidad objetivo → épica → funcionalidad → evidencia.

i Fuente — Las "épicas" (E1–E7) son una agrupación temática utilizada en este informe para organizar el contenido; no son un artefacto nativo de GitHub. Los 7 milestones reales del repositorio (M08) se organizan de forma cronológica por fase de entrega ("Phase 1.A: Auth & Onboarding", "Phase 1.B: Search & Filtering", "Phase 2: Booking & Payments", "Phase 2.B: Notifications", "Phase 3: Host Features", "Phase 3.B: Admin Panel", "Phase 2+: Polish & Optimization"), no por temática funcional. La correspondencia detallada entre épica, milestone e issues se documenta en la sección 09 (Planificación Scrum, Tabla 18) y en [Anexo III — Backlog Completo de User Stories](#) (Tabla 51).

Esta trazabilidad explícita —de objetivo a épica, funcionalidad y evidencia— es en sí misma una forma de cumplir OE6: cada afirmación de este documento remite a un artefacto verificable, ya sea un archivo del repositorio, una métrica de la nota Datos-Verificables o un issue del repositorio de GitHub. Los objetivos aquí definidos se contrastan con el problema que les da origen en [Problema a Resolver](#), y el balance entre lo planificado y lo entregado se retoma en [Conclusiones](#).

05. Problema a Resolver

Problema central

Buscar, comparar y reservar un salón de eventos en la provincia de Tucumán depende, en la actualidad, de canales informales y dispersos —recomendaciones personales, publicaciones en redes sociales, llamados telefónicos— sin un canal único que permita comparar disponibilidad, precio y condiciones entre distintas opciones antes de comprometerse con una reserva. Este problema afecta a los dos perfiles de usuario de forma distinta pero relacionada: al organizador, porque no cuenta con una forma sistemática de evaluar alternativas ni de conocer el precio real de un salón sin contactarlo directamente; y al propietario del salón (el anfitrión), porque no dispone de un canal propio de visibilidad comercial y debe gestionar cada reserva de forma manual, típicamente por WhatsApp, redes sociales o llamadas telefónicas, sin un registro centralizado del estado de cada una.

Consecuencias para el organizador

Para el organizador de un evento, la ausencia de un catálogo centralizado se traduce en un costo de búsqueda elevado: para conocer siquiera las opciones disponibles en una zona o rango de precio determinado, debe recurrir a múltiples fuentes dispersas —grupos de redes sociales, recomendaciones de conocidos, búsquedas genéricas— sin garantía de estar viendo el universo real de salones disponibles. A esto se suma la falta de transparencia de precios: la mayoría de los salones no publica un precio de referencia, por lo que el organizador debe iniciar una conversación individual con cada opción antes de poder comparar presupuestos, lo que multiplica el tiempo invertido en la etapa de decisión y dificulta descartar alternativas tempranamente.

Consecuencias para el anfitrión

Para el propietario de un salón, la falta de un canal de visibilidad especializado limita su alcance a la red de contactos existente o a la inversión en publicidad genérica en redes sociales, sin un espacio donde competir en igualdad de condiciones frente a salones con mayor trayectoria. La gestión de las reservas recibidas, además, ocurre por canales no estructurados —mensajería instantánea, llamadas telefónicas, planillas manuales— lo que incrementa el riesgo de errores de coordinación, como aceptar dos reservas para la misma fecha y horario, o perder el registro de una solicitud que no llegó a confirmarse.



Figura 4 — Árbol de problemas: causas → problema central → efectos.

Problema, consecuencia y respuesta del sistema

Problema	Consecuencia	Respuesta del sistema
No existe un catálogo centralizado de salones	El organizador dedica tiempo a buscar en múltiples redes sociales y grupos	Catálogo público con búsqueda y filtros en /salones
La disponibilidad de un salón no es visible de antemano	Se coordinan fechas por mensajería y, en ocasiones, se descubre el salón ya ocupado	Bloqueos de disponibilidad definidos por el anfitrión y validados en el wizard de reserva
No hay comparación de precio ni de condiciones entre salones	El organizador no puede estimar presupuesto sin contactar a cada salón por separado	Precio visible por salón (fijo, estimado o a cotizar) y cotización explícita por reserva
La gestión de reservas del anfitrión es manual (WhatsApp, teléfono, planillas)	Riesgo de doble reserva y de pérdida de mensajes o solicitudes	Panel del anfitrión con calendario y gestión del estado de cada reserva
Los salones nuevos o pequeños tienen baja visibilidad comercial	Dependencia casi exclusiva del boca a boca para conseguir clientes	Plan de suscripción "Destacado" que mejora la posición del salón en el catálogo

Tabla 8 — Problema, consecuencia y respuesta del sistema.

Puntos de dolor por actor y alternativas actuales

Actor	Punto de dolor	Alternativa actual
Organizador	No sabe qué salones existen ni su disponibilidad real	Preguntar a conocidos o buscar publicaciones en redes sociales

Actor	Punto de dolor	Alternativa actual
Organizador	No puede comparar precio y condiciones entre salones sin contactarlos uno por uno	Contactar cada salón individualmente por WhatsApp o teléfono
Anfitrión	Baja visibilidad frente a salones con más trayectoria o presencia digital	Publicidad en redes sociales propias y recomendación boca a boca
Anfitrión	Gestión manual de la disponibilidad y de las reservas recibidas	Agenda física, planillas de cálculo o hilos de mensajería

Tabla 9 — Puntos de dolor por actor y alternativas actuales.

⚠ **Dato simulado SIM-02 — Puntos de dolor sin medición directa** Los puntos de dolor de la Tabla 9 se formulan de manera plausible a partir del propio dominio del problema y de las funcionalidades que el producto efectivamente prioriza (ver [Objetivos](#)), y no a partir de una encuesta o entrevista documentada con organizadores o propietarios reales. No deben interpretarse como resultados de una investigación de usuarios formal.

Las respuestas concretas que Hosty da a cada uno de estos puntos se retoman, en términos de beneficio percibido, en [Impacto de la Solución](#), y se contrastan con los objetivos específicos definidos en [Objetivos](#).

06. Impacto de la Solución

Beneficios por tipo de usuario

Para el organizador de eventos, Hosty concentra en un único lugar las tres etapas que antes requerían canales distintos y descoordinados: la búsqueda de opciones, la comparación entre ellas y la reserva propiamente dicha. En lugar de contactar salón por salón para conocer precio y disponibilidad, el organizador filtra el catálogo público, compara alternativas con precio visible y confirma una reserva mediante un flujo guiado que valida la disponibilidad antes de aceptarla. Este cambio es particularmente relevante en la etapa de decisión, donde la posibilidad de descartar alternativas sin necesidad de una conversación individual reduce la fricción del proceso completo de reserva.

Para el anfitrión, el beneficio principal es doble: por un lado, un panel de gestión centralizado que reemplaza la coordinación manual de reservas por una vista única de calendario, estado de cada solicitud y cotización de precio por evento; por otro, una vía de visibilidad comercial adicional a sus canales propios, tanto por aparecer en los resultados de búsqueda del catálogo público como, opcionalmente, mediante el plan de suscripción "Destacado", que mejora su posición dentro de los resultados. Ambos efectos son más relevantes cuanto menor es la trayectoria previa del salón, ya que reducen su dependencia de canales de visibilidad que requieren tiempo o inversión constante para sostenerse.

Qué cambia respecto de la situación anterior

El cambio central que introduce Hosty es el pasaje de una coordinación manual y dispersa —apoyada en WhatsApp, Instagram y el boca a boca— a un flujo digital centralizado, donde tanto la oferta (el catálogo de salones) como la demanda (la reserva y sus estados) quedan registradas en un mismo sistema, consultable por ambas partes. Este pasaje no es meramente cosmético: implica que la disponibilidad de un salón, antes conocida sólo por su propietario, pasa a ser una condición verificable por el sistema antes de aceptar una reserva, lo que reduce el margen de error humano en la coordinación.

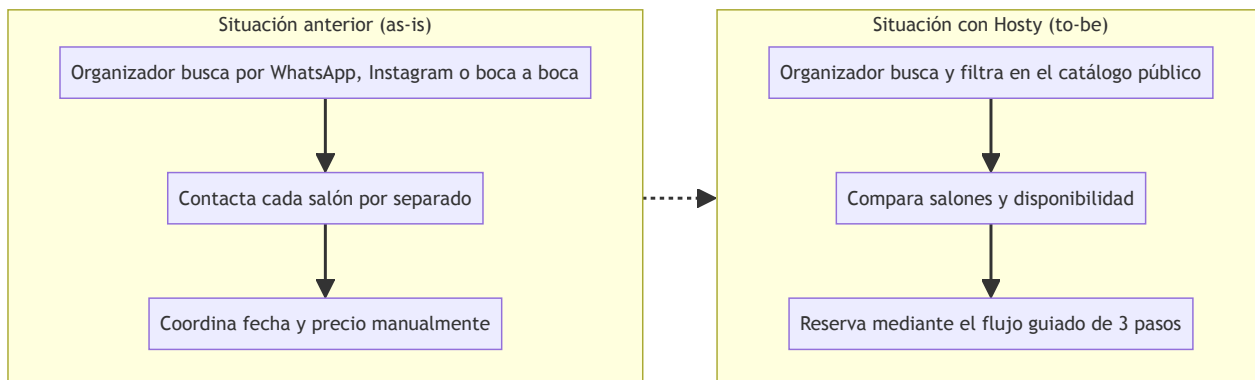


Figura 5 — Proceso as-is (WhatsApp/Instagram/boca a boca) vs. to-be con Hosty.

Dimensión	Tipo de usuario	Impacto esperado	Indicador propuesto	Método de medición
Tiempo de búsqueda	Organizador	Reducción del tiempo dedicado a buscar y comparar salones	Tiempo entre el inicio de la búsqueda y la reserva confirmada	No instrumentado — requiere analítica de producto
Transparencia de precio	Organizador	Precio o rango visible antes de contactar al salón	Porcentaje de salones con precio fijo o estimado publicado	Consulta directa a la tabla <code>salones</code>
Visibilidad comercial	Anfitrión	Mayor exposición del salón dentro del catálogo	Suscripciones activas al plan Destacado	Consulta a <code>salon_subscriptions</code> (M10)
Centralización de la gestión	Anfitrión	Reservas gestionadas desde un único panel en lugar de canales externos	Reservas creadas y actualizadas desde el panel del anfitrión	No instrumentado — requiere adopción real por parte de los anfitriones

Tabla 10 — Impacto por dimensión y tipo de usuario, con indicador y método de medición.

⚠ **Dato simulado SIM-03 — Indicadores de impacto propuestos, no medidos** Los indicadores y métodos de medición de la Tabla 10 son propuestas razonables para evaluar el impacto de la solución, pero el proyecto no cuenta, a la fecha de este informe, con instrumentación de analítica de producto que permita reportarlos como datos reales. Dos de las cuatro filas sí se apoyan en datos verificables del modelo de datos (M10); las otras dos quedan explícitamente señaladas como no instrumentadas.

El impacto aquí descrito retoma directamente los puntos de dolor identificados en [Problema a Resolver](#) y se refleja, en términos cuantitativos, en las métricas de [Métricas](#).

07. Equipo y Roles

Composición del equipo

El proyecto fue desarrollado por un equipo de 5 integrantes, identificados de forma consolidada a partir de 9 identidades Git distintas (M05): Juan Pablo Valdez, Juan Ignacio Mignone, Lautaro Naglieri, Benjamín Garma y Pablo Czurylo. La distribución de roles de equipo bajo el marco Scrum adoptado —1 Product Owner, 1 Scrum Master y 3 desarrolladores, uno de ellos con foco en calidad (QA)— se infiere de la actividad observable en el historial de commits, ya que el proyecto no cuenta con un registro documental explícito de la asignación formal de roles. Estas responsabilidades reflejan, ante todo, el área funcional donde cada integrante concentró su trabajo a lo largo del proyecto, verificable directamente en el historial de commits del repositorio, y no necesariamente una asignación fija o exclusiva: la naturaleza de un equipo de 5 personas trabajando sobre un mismo repositorio implica solapamientos razonables entre áreas.

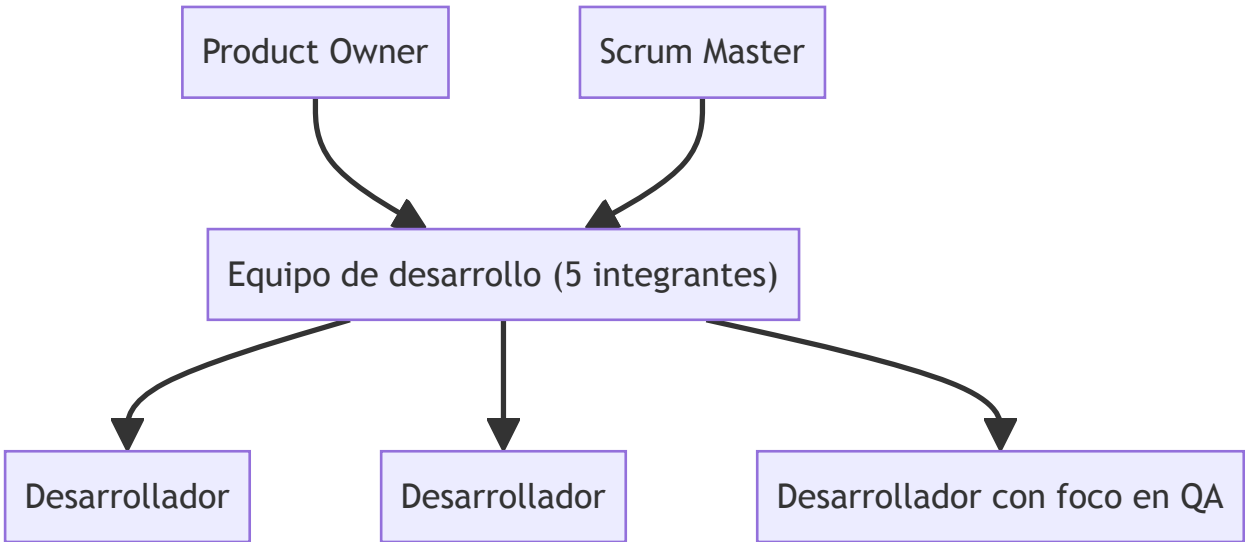


Figura 6 — Organigrama Scrum: PO / SM / equipo de desarrollo (5 integrantes).

Integrante	Rol de equipo (Scrum)	Responsabilidades principales
Juan Pablo Valdez	Product Owner	Arquitectura general; catálogo de salones, panel del anfitrión, flujo de reserva y autenticación — mayor volumen de contribuciones del equipo
Juan Ignacio Mignone	Scrum Master	Catálogo de salones, página de inicio, panel del anfitrión y favoritos
Lautaro Naglieri	Desarrollador	Catálogo de salones, panel del anfitrión y flujo de reserva
Benjamín Garma	Desarrollador con foco en QA	Infraestructura de pruebas (Vitest, Playwright, Cypress) y el workflow de CI <code>frontend-tests.yml</code>
Pablo Czurylo	Desarrollador	Búsqueda de salones, flujo de reserva y notificaciones por email

Tabla 11 — Integrantes, rol de equipo y responsabilidades.

⚠ **Dato simulado SIM-04 — Asignación de rol de equipo** La columna "Rol de equipo (Scrum)" es una reconstrucción plausible a partir del volumen y del área de los commits de cada integrante, no un registro documentado de la asignación real de roles. La única excepción es el foco en QA de Benjamín Garma, que se verifica directamente en los archivos de configuración de pruebas (Vitest, Playwright, Cypress) y en el workflow de CI que aportó al repositorio.

✓ **PLACEHOLDER P-09 — Rol formal de cada integrante** Completar, para la Tabla 11, el rol formal de cátedra de cada integrante (no el rol de equipo Scrum de la fila anterior) y sus legajos — el mismo dato pendiente ya señalado como P-06 en la sección 00 (Tabla 1). Responsable: equipo.

Contribuciones por identidad Git

Integrante	Commits (todas las identidades)	Porcentaje del total
Juan Pablo Valdez	140	59,3 %
Juan Ignacio Mignone	45	19,1 %
Lautaro Naglieri	33	14,0 %
Benjamín Garma	10	4,2 %
Pablo Czurylo	8	3,4 %

Tabla 12 — Contribuciones por identidad Git.

📄 **Fuente — M05 / M02:** `git shortlog -sne --all`. El detalle de cada identidad Git por integrante se documenta en [_\(ver documentacion-final/meta/Datos-Verificables.md\)](#); esta tabla sólo consolida el porcentaje sobre el total de 236 commits (M02).

Roles de usuario, permisos y mecanismo de autorización

A diferencia de los roles de equipo descriptos arriba, Hosty **no tiene una tabla de roles de usuario ni un esquema de control de acceso basado en roles (RBAC)**. La autorización se resuelve exclusivamente por propiedad (*ownership*): las políticas de seguridad a nivel de fila (RLS) de Postgres restringen qué filas puede leer o modificar cada usuario autenticado según `auth.uid()`, sin que exista un campo de rol que se consulte para decidir un permiso. Este enfoque implica que ser anfitrión no es un estado que se activa mediante un campo de configuración o un permiso otorgado por un administrador, sino una consecuencia directa de poseer al menos un registro en la tabla `salones`: cualquier usuario autenticado puede convertirse en anfitrión publicando un salón, sin necesidad de una aprobación previa.

Perfil	Cómo se determina	Permisos principales	Mecanismo de autorización
Visitante anónimo	No autenticado	Ver el catálogo público y el detalle de cada salón	Ninguno — datos públicos vía políticas RLS de lectura
Usuario autenticado (organizador)	Sesión activa de Supabase Auth	Reservar, marcar favoritos, ver sus propias reservas	RLS: filas visibles o editables según <code>auth.uid()</code>
Anfitrión	Implícito: posee al menos una fila en <code>salones</code>	Gestionar sus salones, ver y responder reservas recibidas, suscribirse al plan Destacado	RLS: acceso restringido a filas de <code>salones / bookings</code> donde el propietario coincide con <code>auth.uid()</code>
Administrador	No implementado	—	— (ver hallazgo a continuación)

Tabla 13 — Roles de usuario, permisos y mecanismo de autorización.

Esta decisión de diseño simplifica el modelo de permisos del sistema, a costa de no contar con una jerarquía de administración: cualquier limitación operativa (por ejemplo, moderar un salón inapropiado) requiere hoy una intervención manual directa sobre la base de datos, dado que no existe un perfil de administrador funcional en la aplicación.

¡ Fuente — El issue #46 ("feat(admin): panel de aprobación y moderación de salones") figura cerrado en el repositorio, pero no existe en el código ninguna ruta, componente o columna de aprobación/moderación asociada (`grep` sobre `frontend/src/` y `supabase/migrations/*.sql` sin resultados): el panel de administrador no fue implementado pese al cierre del issue. La cadena de autorización por propiedad se ilustra gráficamente en el Anexo I (Figura 29).

La composición y las responsabilidades del equipo descritas aquí se retoman, en clave de proceso Scrum, en [Planificación Scrum](#); el detalle técnico de la arquitectura de autorización se desarrolla en [Arquitectura](#).

08. Diseño y Desarrollo

Esta sección describe el proceso de diseño seguido, el inventario funcional de pantallas organizado por tipo de usuario, la arquitectura de carpetas que materializa esas pantallas en código, y las decisiones de UX/UI adoptadas junto con su justificación. Los diagramas de flujo de cada proceso de negocio (búsqueda, reserva, publicación, estados de una reserva, favoritos) se documentan en detalle en [Anexo II — Diagramas de Flujo Complementarios](#) para evitar duplicación, y la justificación arquitectónica de esta organización de carpetas se profundiza en [Arquitectura](#).

8.1 Proceso de diseño

El desarrollo de la interfaz siguió un proceso iterativo de cinco etapas, sin una fase de diseño visual centralizada previa a la implementación: relevamiento de requerimientos por módulo, wireframes de baja fidelidad, definición del sistema de diseño (tokens de marca en `frontend/src/index.css`), implementación directa con componentes de `shadcn/ui`, y revisión funcional antes de cada entrega. Las etapas de design system e implementación se retroalimentaron de forma iterativa a medida que se incorporaron nuevos módulos (reservas, panel de anfitrión, plan destacado).

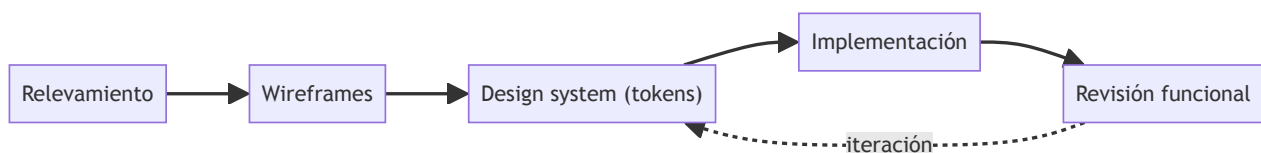


Figura 7 — Proceso de diseño: relevamiento → wireframes → design system → implementación → revisión.

8.2 Módulos y pantallas por tipo de usuario

El inventario de pantallas se organiza en tres tipos de usuario, coherentes con la aclaración de [Equipo y Roles](#) de que Hosty no tiene una tabla de roles: "anfitrión" es una condición derivada de poseer al menos un registro propio en `salones`, no un rol almacenado.

Visitante (sin sesión). La Home (`/`) presenta salones destacados (`useFeaturedSalones`); `/salones` ofrece búsqueda con filtros y alterna entre vista de lista y vista de mapa (Leaflet); `/salones/$id` muestra el detalle completo de un salón (fotos, precio, servicios y disponibilidad). `/login` y `/register` completan el acceso.

Usuario autenticado (organizador). `/mis-reservas` lista las reservas propias con su estado; `/mis-favoritos` lista los salones marcados; `/mi-perfil` permite actualizar nombre y contraseña; `/salones/$id/reservar` implementa el flujo de reserva guiado de tres pasos.

Anfitrión. `/host/dashboard` centraliza los salones publicados y las reservas recibidas; `/host/create` y `/host/$id/edit` exponen el mismo asistente de publicación de cuatro pasos, en modo creación y edición respectivamente; `/host/$bookingId` permite revisar, cotizar, confirmar o rechazar una reserva puntual.

Ruta	Componente	Tipo de usuario
<code>/</code>	<code>HomePage</code>	Visitante
<code>/salones</code>	<code>Layout (Outlet)</code>	Visitante
<code>/salones/</code>	<code>SalonesPage</code>	Visitante
<code>/salones/\$id</code>	<code>SalonDetailPage</code>	Visitante
<code>/login</code>	<code>LoginPage</code>	Visitante
<code>/register</code>	<code>RegisterPage</code>	Visitante
<code>/salones/\$id/reservar</code>	<code>BookingFlow</code>	Usuario autenticado
<code>/mis-reservas</code>	<code>MyBookingsPage</code>	Usuario autenticado
<code>/mis-favoritos</code>	<code>MisFavoritosPage</code>	Usuario autenticado
<code>/mi-perfil</code>	<code>MiPerfilPage</code>	Usuario autenticado
<code>/host/dashboard</code>	<code>HostDashboardPage</code>	Anfitrión
<code>/host/create</code>	<code>CreateSalonPage</code>	Anfitrión
<code>/host/\$id/edit</code>	<code>EditSalonPage</code>	Anfitrión
<code>/host/\$bookingId</code>	<code>BookingDetailPage</code>	Anfitrión

Tabla 15 — Inventario de pantallas: ruta, componente y tipo de usuario.

La ruta `/salones` es un caso particular: no renderiza una pantalla propia, sino un `Outlet` de TanStack Router que agrupa `/salones/`, `/salones/$id` y `/salones/$id/reservar` bajo un mismo segmento de URL. Se incluye en el inventario porque el comando de conteo de M09 la contabiliza como archivo de ruta, aun cuando no tiene contenido visual propio.

❗ **Fuente — M09: 14 rutas, 6 públicas / 8 protegidas con `requireAuth` (`frontend/src/routes/`).**

8.3 Mapa de navegación

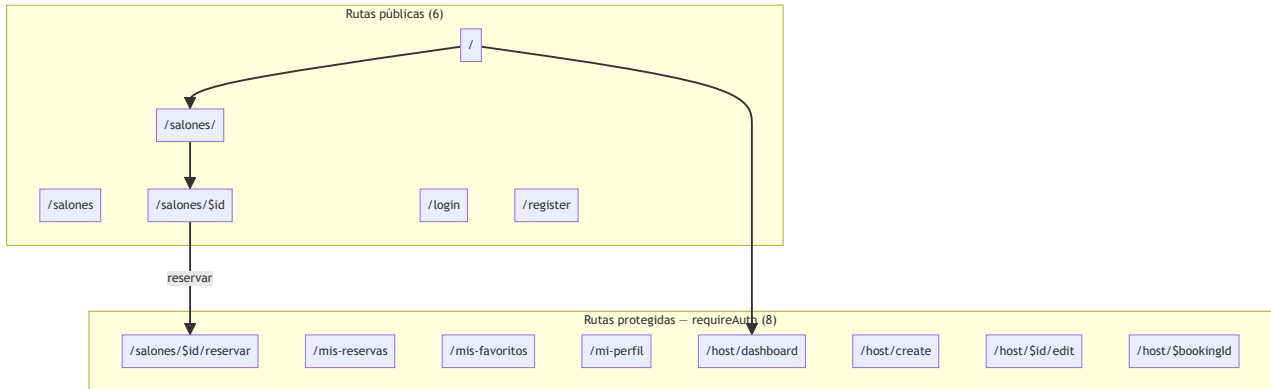


Figura 8 — Mapa de navegación: 14 rutas — 6 públicas / 8 protegidas (`requireAuth`).

8.4 Anatomía de una feature

El código de cada módulo funcional sigue una organización por *feature*, no por tipo técnico de archivo: cada carpeta bajo `features/<nombre>/` agrupa sus propios `components/`, `api/` (hooks de TanStack Query sobre `supabase-js`) y `types.ts`. Las rutas de `routes/` son deliberadamente delgadas — sólo declaran el path, la validación de búsqueda (Zod) y, cuando corresponde, el guard `requireAuth` — y delegan toda la lógica visual y de datos en el componente de la feature correspondiente.

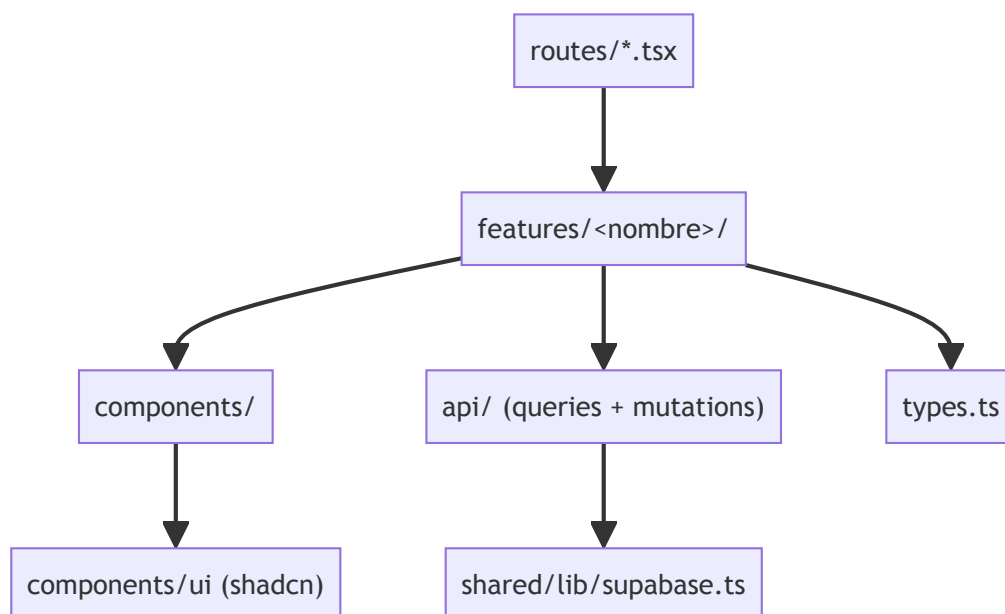


Figura 9 — Anatomía de una feature: `features/<n>/{components,api,types.ts}` ↔ `routes/` ↔ `shared/lib`.

8.5 Decisiones de UX/UI

La paleta e identidad visual provienen del docs/hosty-brandbook.pdf (v1.0, abril 2026): el isotipo combina la letra "H" con un arco —la forma de una entrada o umbral— y un punto central que representa un pin de ubicación, coherente con un producto de búsqueda geolocalizada de salones.

Token	Valor	Uso
--color-coral	#E8452A	Acción principal (primary)
--color-ink	#1C2B3A	Texto y jerarquía visual (foreground)
--color-bone	#FAF8F5	Fondo cálido (background)
--color-amber	#F5A623	Acento cálido (accent)
--font-sans	Plus Jakarta Sans	Tipografía principal (encabezados y cuerpo)
--font-serif-accent	Instrument Serif	Acento tipográfico puntual

Tabla 14 — Tokens de marca y tipografía.

¡ Fuente — docs/hosty-brandbook.pdf **(págs. 4, 8-9); frontend/src/index.css (bloque @theme).**

Decisión	Justificación
Tokens de marca vía Tailwind v4 CSS-first (@theme)	Sin configuración JS separada; un único punto de verdad para color y tipografía
Diseño mobile-first	La búsqueda y reserva de un salón ocurren mayormente desde el celular
shadcn/ui sobre primitivos Radix	Componentes accesibles con código propio, sin dependencia de un paquete de UI cerrado
Búsqueda con mapa (Leaflet + Nominatim)	Experiencia geolocalizada específica de Tucumán sin costo de licencia de mapas
Asistentes ("wizards") de varios pasos para reserva (3) y publicación (4)	Reduce la carga cognitiva de formularios largos y permite validar por etapas
Iconografía outline uniforme (lucide-react , strokeWidth=1.5)	Coherente con el pack de íconos outline definido en el brandbook

Tabla 16 — Decisiones de UX/UI y su justificación.

La escala tipográfica de encabezados también proviene del sistema de diseño: h1 usa peso 800, h2 peso 700 y h3 / h4 peso 600, todos sobre la misma familia Plus Jakarta Sans, replicando la jerarquía definida en el brandbook (H1/H2/H3 en la sección de tipografía) en lugar de introducir pesos ad hoc por componente. El modo oscuro reutiliza los mismos tokens semánticos (--background , --foreground , etc.) con valores HSL alternativos, de modo que ningún componente necesita lógica condicional de tema: sólo cambia la clase dark en el elemento raíz.

8.6 Diagramas de flujo principales

El detalle diagramado de cada proceso de negocio —búsqueda y filtrado, reserva guiada, publicación de un salón, máquina de estados de una reserva y gestión de favoritos/plan destacado— se documenta como Figuras 30 a 34 en [Anexo II — Diagramas de Flujo Complementarios](#), para mantener en esta sección únicamente el diseño de la interfaz y no duplicar diagramas de proceso.

09. Planificación Scrum

El proyecto se organizó bajo el marco Scrum, con iteraciones quincenales y un backlog gestionado íntegramente como issues de GitHub, agrupadas en épicas y priorizadas mediante un tablero de gestión visual. Esta sección documenta las épicas, las historias de usuario destacadas, los criterios de aceptación, las reglas de trabajo del equipo, el calendario de sprints y el mecanismo de seguimiento utilizado.

Ceremonias y cadencia

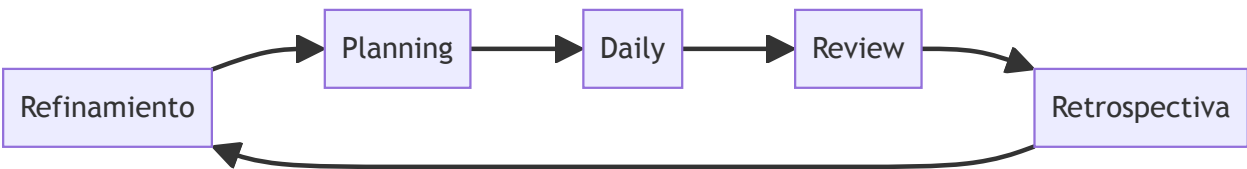


Figura 10 — Iteración Scrum: refinamiento, planificación, daily, revisión y retrospectiva.

⚠ **Dato simulado SIM-09 — Ceremonias Scrum y cadencia** No existe un acta formal de ceremonias en el repositorio. La Tabla 17 que aparece a continuación (ceremonias Scrum y cadencia) reconstruye de forma plausible la cadencia y los participantes para un equipo estudiantil de cinco integrantes que trabaja con Scrum sobre issues de GitHub.

Ceremonia	Frecuencia	Participantes	Propósito
Refinamiento	Semanal	Equipo completo	Detallar y estimar issues antes del siguiente sprint
Planning	Inicio de cada sprint	Equipo completo	Seleccionar y comprometer el alcance del sprint
Daily	Diaria (15 min)	Equipo de desarrollo	Sincronizar avance y destrabar bloqueos
Review	Cierre de cada sprint	Equipo + Product Owner	Demostrar el incremento funcional
Retrospectiva	Cierre de cada sprint	Equipo completo	Identificar mejoras de proceso

Tabla 17 — Ceremonias Scrum y cadencia.

Épicas

El backlog se organiza en siete épicas, correspondidas con los siete hitos (*milestones*) reales del repositorio (M08). La correspondencia se estableció por afinidad temática de las issues que integra cada hito; el detalle issue por issue se documenta en [Anexo III — Backlog Completo de User Stories \(T51\)](#), donde cada fila cita el hito real de GitHub sin pasar por esta simplificación en siete categorías.

Épica	Objetivo	Hito(s) de GitHub asociados	Estado
E1 — Catálogo y búsqueda	Catálogo público con búsqueda, filtros y geolocalización	Phase 1.B: Search & Filtering; Phase 3.B: Admin Panel (moderación de publicaciones)	Completada (7/7 issues)
E2 — Autenticación y cuenta	Registro, login y alta de cuenta de organizador/anfitrión	Phase 1.A: Auth & Onboarding	Completada (3/3)
E3 — Reserva de salones	Flujo de reserva guiado y cobro de comisión	Phase 2: Booking & Payments	En curso (3/4; #45 abierta)
E4 — Panel del anfitrión	Gestión de reservas, precios, agenda y notificaciones	Phase 3: Host Features; Phase 2.B: Notifications	Completada (6/6)
E5 — Favoritos y plan destacado	Favoritos del organizador y suscripción destacada del anfitrión	Sin hito propio (issue #47 + issues #75/#88 sin hito)	Completada (3/3)
E6 — Calidad e integración continua	Pruebas automatizadas y estabilización del pipeline de CI	Subconjunto de Phase 2+: Polish & Optimization	Completada (4/4)
E7 — Infraestructura y despliegue	Infraestructura de producción y optimización de rendimiento	Subconjunto de Phase 2+: Polish & Optimization	En curso (3/4; #35 abierta)

Tabla 18 — Épicas: código, objetivo y estado.

i Fuente — M06 (issues totales/cerradas), M08 (7 milestones): `gh issue list --state all , gh api .../milestones (verificado 2026-07-28); ver _ (ver documentacion-final/meta/Datos-Verificables.md).`

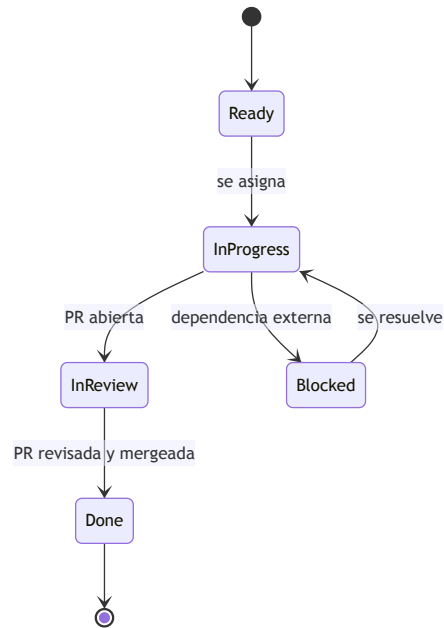


Figura 11 — Ciclo de vida de un issue en GitHub Projects v2: Todo → In Progress → In Review → Done (+ Blocked).

¡ Fuente — M19: distribución real de estados en el tablero #4 al momento de la verificación — Done: 45, Ready: 4, In review: 1 (gh project item-list 4 --owner juanpablovaldez --format json , 2026-07-28). Los estados "In Progress" y "Blocked" existen en el esquema del tablero pero no tienen issues asignadas actualmente.

✓ PLACEHOLDER P-15 — Captura del tablero de gestión Guardar la imagen como assets/f12-tablero-projects.png y reemplazar este bloque por !f12-tablero-projects.png . Responsable: equipo. Destino: Figura 12.

Figura 12 — Tablero de gestión del proyecto en GitHub Projects v2 (board #4).

User stories destacadas

La siguiente selección de 15 historias, sobre un total de 50 issues (M06), cubre las siete épicas. Los story points citados no son una estimación de este informe: corresponden al campo "Size" registrado por el propio equipo en el tablero de GitHub Projects v2, con escala de Fibonacci (1, 2, 3, 5, 8, 13, 21).

Historia	Como / quiero / para	SP	Épica
#13	Como usuario, quiero registrarme e iniciar sesión para acceder a mi cuenta y a las funciones protegidas	13	E2
#18	Como anfitrión, quiero un panel para ver y administrar mis salones publicados	13	E2
#19	Como anfitrión, quiero un asistente guiado de publicación para cargar un salón sin errores	21	E2
#11	Como organizador, quiero buscar y filtrar salones para encontrar opciones acordes a mi evento	13	E1

Historia	Como / quiero / para	SP	Épica
#15	Como organizador, quiero ver el detalle de un salón con galería y características para decidir si reservarlo	13	E1
#30	Como organizador, quiero paginación o scroll infinito en el listado para explorar más salones	8	E1
#16	Como organizador, quiero seleccionar fecha y horario disponibles para reservar sin conflictos	21	E3
#17	Como organizador, quiero confirmar mi reserva y verla en mi panel para hacer seguimiento de su estado	13	E3
#31	Como organizador, quiero completar el flujo de reserva de punta a punta para concretar el alquiler	21	E3
#65	Como anfitrión, quiero confirmar o rechazar reservas recibidas para controlar la ocupación de mi salón	13	E4
#66	Como anfitrión, quiero definir precios flexibles y servicios adicionales para ajustar mi oferta	8	E4
#67	Como anfitrión, quiero bloquear fechas en un calendario para evitar reservas en días no disponibles	5	E4
#47	Como anfitrión, quiero suscribirme a un plan destacado para aumentar la visibilidad de mi salón	13	E5
#21	Como equipo de desarrollo, quiero pruebas end-to-end del flujo de reserva para validar el camino crítico	13	E6
#22	Como equipo de desarrollo, quiero infraestructura de producción con CDN y HTTPS para publicar con seguridad	8	E7

Tabla 19 — User stories destacadas: formato Como/quiero/para, story points y épica.

i Fuente — M25: story points reales del campo "Size" del tablero #4 — `gh project item-list 4 --owner juanpablovaldez --format json` (verificado 2026-07-28); 40 de las 50 issues del backlog tienen el campo cargado. Ver [_](#)(ver documentacion-final/meta/Datos-Verificables.md).

Ejemplo de criterio de aceptación

△ **Dato simulado SIM-12** — Ejemplo de criterio de aceptación (formato Given/When/Then) El issue original no registra sus criterios de aceptación en formato Given/When/Then. La redacción siguiente se reconstruye a partir del comportamiento observable en `BookingFlow.tsx` y `useCreateBooking` para ilustrar el método de trabajo del equipo, sin constituir evidencia documental del proceso real.

Historia: Como organizador, quiero reservar un salón en una fecha y horario disponibles (#16, #31).

- **Given** un salón publicado con disponibilidad para el rango de fechas solicitado.

- **When** el organizador completa el asistente de reserva (fecha/horario, datos del evento, confirmación) y no existe superposición con un bloqueo de disponibilidad ni con otra reserva confirmada.
- **Then** el sistema crea la reserva con estado `pending` y la expone en el panel del organizador y en el panel del anfitrión para su revisión.

Definition of Ready y Definition of Done

△ **Dato simulado SIM-10 — Definition of Ready (DoR)** No hay un documento de DoR versionado en el repositorio. Las filas "DoR" de la Tabla 20 que aparece a continuación (Definition of Ready y Definition of Done) reconstruyen un criterio plausible para un equipo estudiantil de cinco integrantes.

△ **Dato simulado SIM-11 — Definition of Done (DoD)** Ídem SIM-10: las filas "DoD" de la misma Tabla 20 son una reconstrucción plausible, no un acta registrada del equipo.

Tipo	Criterio
DoR	La historia tiene una descripción clara, un criterio de aceptación esbozado y no depende de otra historia sin resolver
DoR	La historia está estimada en story points y priorizada en el tablero antes del planning
DoD	El código pasa lint, typecheck y la suite de pruebas automatizadas (M12)
DoD	La funcionalidad fue revisada por al menos un integrante distinto del autor (pull request)
DoD	La historia se verificó manualmente contra su criterio de aceptación antes de cerrarse

Tabla 20 — Definition of Ready y Definition of Done.

Plan de sprints

△ **Dato simulado SIM-13 — Límites y foco de los sprints** El equipo no registró formalmente los sprints como "S1"-"S5". En la Tabla 21 que aparece a continuación (plan de sprints), los límites de fecha y el foco de cada sprint se infieren a partir de la densidad real de commits y de los clústeres de fecha de las migraciones de Supabase; los conteos de commits e issues cerradas por sprint sí son reales y verificables (M23, M24).

Sprint	Rango	Foco	Commits	Issues cerradas	Decisión / resultado
S1	2026-03-29 – 2026-04-19	Arranque, arquitectura base, home	50	0	Scaffolding inicial; sin cierres formales de issues aún

Sprint	Rango	Foco	Commits	Issues cerradas	Decisión / resultado
S2	2026-04-20 – 2026-05-15	Catálogo, filtros, detalle de salón	38	4	Migración de NestJS a Supabase (commit 3a89616)
S3	2026-05-16 – 2026-06-05	Autenticación, carga de imágenes (Storage)	39	16	Mayor concentración de cierres del proyecto
S4	2026-06-06 – 2026-06-13	Panel de anfitrión, reservas, precios y bloqueos	13	12	Corrección del CHECK de bookings a 4 estados
S5	2026-06-14 – 2026-06-24	Plan destacado, coordenadas, favoritos	41	13	Consolidación de 7 PRs en la PR #93

Tabla 21 — Plan de sprints: cantidad, duración, foco y resultado.

! Fuente — M23 (commits por sprint) y M24 (issues cerradas por sprint), calculados para este informe: `git log dev --since --until --oneline | wc -l` y `gh issue list --state closed --json number,closedAt` por ventana (verificado 2026-07-28). Ver [_](#) (ver documentacion-final/meta/Datos-Verificables.md). El detalle cronológico se desarrolla en [Ejecución por Sprint](#).

Herramienta de gestión: el seguimiento del backlog y del avance de cada sprint se realizó en GitHub Projects v2, tablero #4 ("Hosty"), con campos de estado, tamaño (story points) y hito (M19). No se utilizó una herramienta externa (Jira, Trello); el tablero está integrado directamente con las issues y pull requests del repositorio.

Retrospectivas

△ Dato simulado SIM-14 — Retrospectiva S1–S2 · SIM-15 — Retrospectiva S3–S4 · SIM-16 — Retrospectiva S5 No existe acta de retrospectiva registrada. La Tabla 22 que aparece a continuación (retrospectivas) reconstruye de forma plausible el contenido a partir de fricciones observables en el historial (issues de re-trabajo, la corrección del CHECK de bookings , el evento de consolidación de PRs) y no debe interpretarse como transcripción real de una ceremonia.

Sprints	Problema	Impacto	Acción correctiva
S1–S2	Se subestimó el esfuerzo de migrar de NestJS a Supabase	Retrabajo de la capa de acceso a datos a mitad de sprint	Congelar decisiones de arquitectura de backend antes del planning siguiente
S3–S4	Las políticas RLS por tabla se diseñaron de forma reactiva, issue por issue	El estado declined quedó implementado en el frontend antes de habilitarse en el CHECK de la base	Revisar el modelo de datos completo antes de habilitar un nuevo estado de negocio

Sprints	Problema	Impacto	Acción correctiva
S5	Varias ramas de feature quedaron abiertas en simultáneo cerca del cierre	Riesgo de conflictos de integración y de una migración duplicada (20260614000001)	Consolidar las ramas activas en una única PR de integración antes de cerrar el período

Tabla 22 — Retrospectivas: problema, impacto y acción correctiva.

10. Presupuesto

Esta sección presenta una estimación de costos del proyecto bajo dos componentes: el esfuerzo de las personas involucradas (recursos humanos) y la infraestructura tecnológica utilizada. El primer componente es una simulación con supuestos declarados, dado que el equipo no facturó horas reales; el segundo es un dato verificado, ya que el proyecto operó dentro de capas gratuitas durante todo el desarrollo.

Esfuerzo por perfil

⚠ **Dato simulado SIM-17 — Tarifas y dedicación horaria** Las tarifas por hora y la dedicación semanal son una estimación de mercado para perfiles junior/estudiantiles en Tucumán durante 2026; no provienen de una factura o cotización real. La Tabla 23 que aparece a continuación (estimación de esfuerzo por perfil) reagrupa a los mismos cinco integrantes de [Equipo y Roles](#) (Tabla 11) por perfil de costeo, que no coincide necesariamente con el rol Scrum de cada persona.

Perfil	Dedicación semanal	Horas totales (12,6 semanas)	Tarifa (ARS/hora)	Subtotal (ARS)
Product Owner / Scrum Master	5 h	63	8.000	504.000
Desarrollador/a Frontend (1)	15 h	189	10.000	1.890.000
Desarrollador/a Frontend (2)	15 h	189	10.000	1.890.000
QA	8 h	101	9.000	909.000
Diseño UX/UI	6 h	76	9.000	684.000
Subtotal RRHH		618		5.877.000

Tabla 23 — Estimación de esfuerzo por perfil, horas y tarifa.

Infraestructura y capas gratuitas

Servicio	Costo real durante el desarrollo	Estimación de producción comercial
Supabase (Auth, Postgres, Storage)	USD 0 — plan Free	USD ≈ 25/mes — plan Pro
AWS S3 + CloudFront	USD 0 — Free Tier (12 meses)	USD ≈ 10-15/mes — tráfico moderado
Dominio propio	— (no adquirido)	USD ≈ 12/año
Nominatim (geocodificación)	USD 0 — API pública	USD 0 dentro de límites de tasa

Tabla 24 — Costos de infraestructura y capas gratuitas.

❗ **Fuente** — Costo real de infraestructura durante el desarrollo: USD 0, dado que el proyecto operó dentro de las capas gratuitas de Supabase y de AWS (S3 + CloudFront) — `infra/*.tf` (Terraform del proyecto), sin facturación registrada (verificado 2026-07-28).

⚠ **Dato simulado SIM-19** — Estimación de costo de producción comercial En la Tabla 24 anterior (costos de infraestructura y capas gratuitas), los montos de la columna "Estimación de producción comercial" son valores de lista pública de los proveedores al momento de redactar este informe, no una cotización contratada.

Costo total y supuestos

⚠ **Dato simulado SIM-18** — Costo total del proyecto El total de la Tabla 25 que aparece a continuación (costo total, contingencia y supuestos) surge de aplicar los supuestos de la Tabla 23 (simulados) más una contingencia; no refleja un desembolso real, ya que el equipo no facturó honorarios entre sí.

Concepto	Monto (ARS)
Subtotal RRHH	5.877.000
Infraestructura y herramientas (provisión)	50.000
Contingencia (15% sobre RRHH + infraestructura)	889.050
Total estimado	6.816.050

Tabla 25 — Costo total, contingencia y supuestos declarados.

Supuestos declarados: (a) tarifas de mercado junior/estudiantil de Tucumán, 2026, sin facturación real; (b) dedicación part-time compatible con cursada, entre 5 y 15 horas semanales según perfil; (c) duración de 12,6 semanas (M03/M04, 88 días corridos); (d) infraestructura excluida del subtotal de RRHH y presentada por separado (Tabla 24), dado que su costo real fue nulo; (e) contingencia del 15% para cubrir retrabajo e imprevistos de alcance no planificados.

Distribución del presupuesto (ARS)

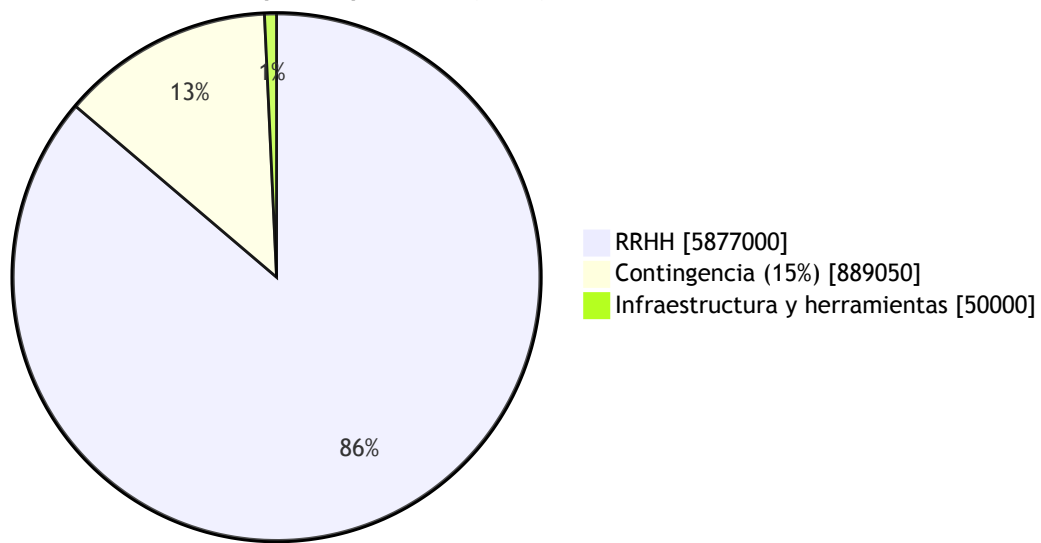


Figura 13 — Distribución del presupuesto por rubro: RRHH, infraestructura y herramientas, contingencia.

△ Dato simulado — ver SIM-18. La distribución de la Figura 13 anterior (gráfico de presupuesto) proviene íntegramente de la Tabla 25, de carácter simulado.

11. Arquitectura

11.1 Patrón arquitectónico

Hosty implementa una **arquitectura cliente-servidor de dos capas basada en BaaS** (*Backend as a Service*): una *single-page application* en React que se comunica directamente con Supabase (PostgREST, Auth, Storage y Postgres con Row Level Security), sin un servidor de aplicación propio intermedio. No se trata de un monolito de tres capas (presentación / lógica de negocio / datos con un backend a medida): la capa de lógica de negocio se reparte entre validación en el cliente (Zod) y restricciones declarativas en la base de datos (`CHECK` , políticas RLS), y la capa de API es generada automáticamente por PostgREST a partir del esquema de Postgres.

Esta decisión es, además, una migración real y no un diseño original: el repositorio contenía un backend NestJS de tres capas (con Prisma sobre PostgreSQL) que fue eliminado por completo en el commit `3a89616` ("refactor: remove entire backend directory and associated CI/CD workflows", 2026-04-29), en favor de Supabase como única capa de datos y autenticación. La justificación observable de ese cambio es la escala del proyecto: un MVP de marketplace no requiere lógica de servidor a medida cuando Postgres + RLS + PostgREST cubren CRUD, autorización por propiedad y generación de API sin código adicional.

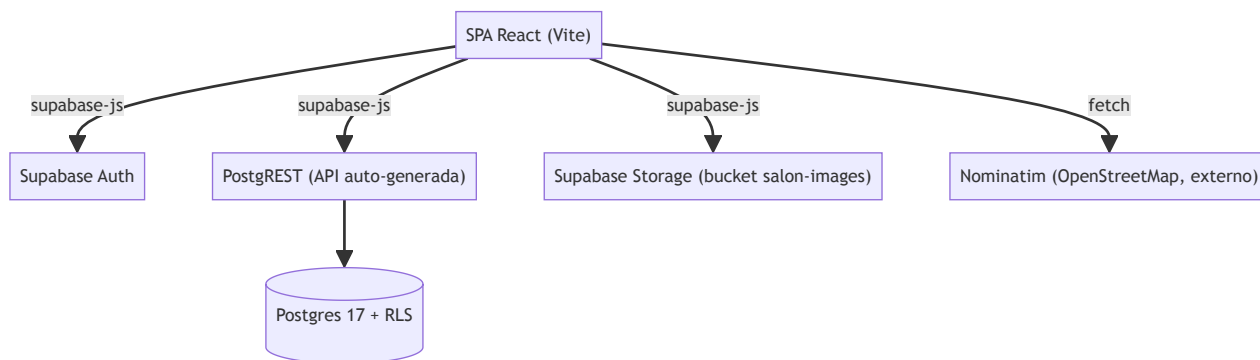


Figura 14 — Arquitectura general: SPA React ↔ Supabase (Auth/PostgREST/Storage/Postgres+RLS).

11.2 Despliegue (visión general)

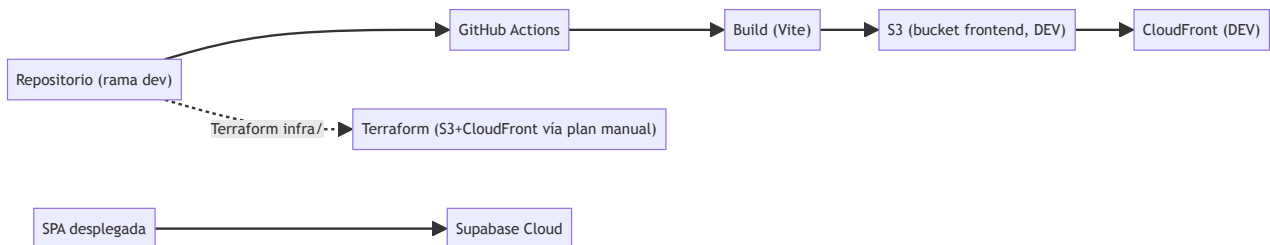


Figura 15 — Despliegue: repo → GitHub Actions → build → S3+CloudFront (DEV); Supabase Cloud; Terraform.

Una vez desplegada, la SPA resuelve su propia sesión de autenticación al arrancar en el navegador, antes de que cualquier ruta protegida decida si continúa o redirige: `main.tsx` invoca `initAuth()`, que resuelve la sesión existente (`getSession()`), la vuelca en `auth.store` (Zustand) y resuelve una promesa módulo `authReady` antes de que cualquier `beforeLoad` / `requireAuth` decida si redirige a `/login`.

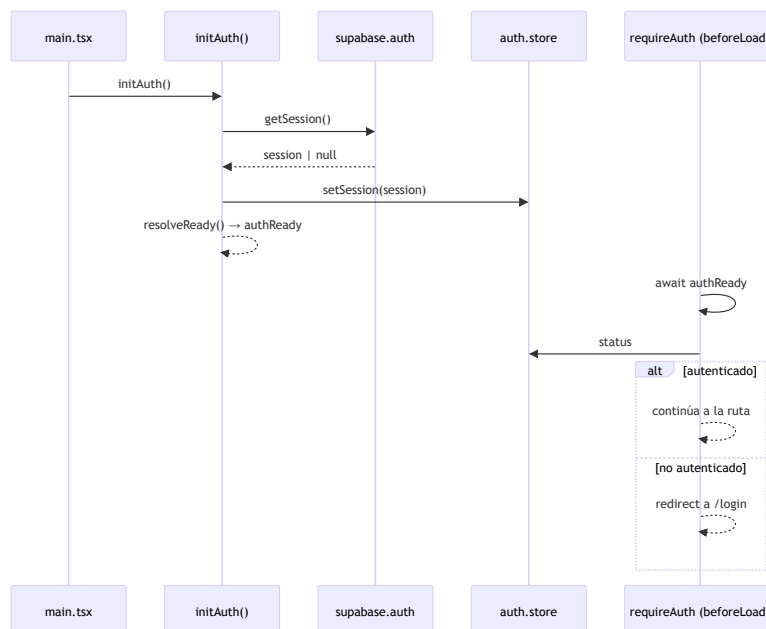


Figura 16 — Bootstrap de autenticación: `main.tsx` → `initAuth()` → `getSession()` → `auth.store` → `authReady` → ruta o redirect.

11.3 Frontend

El frontend es una SPA React 19 servida por Vite, con enrutamiento *file-based* de TanStack Router y estado de servidor manejado por TanStack Query sobre `supabase-js`. La organización de carpetas es por *feature* (ver [Diseño y Desarrollo](#), §8.4): `routes/` sólo declara paths, `validateSearch` y guards; `features/<n>/` concentra componentes, hooks de datos y tipos; y `shared/lib/` aloja el cliente de Supabase y utilidades transversales. La regla de dependencia es estricta en un sentido: una feature nunca importa de otra feature.

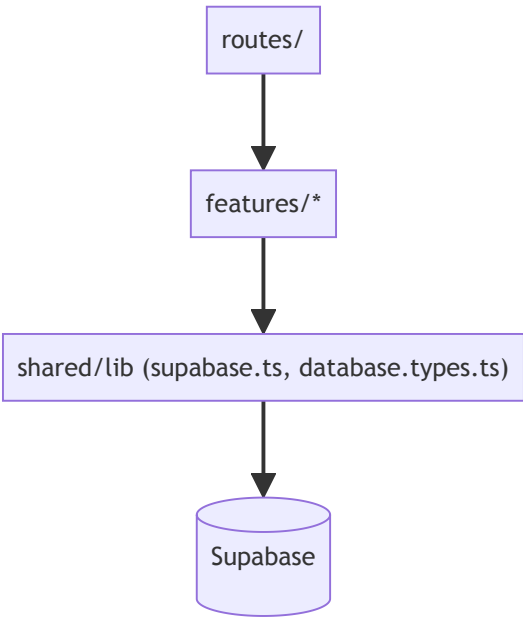


Figura 17 — Capas y dependencias permitidas: `routes` → `features` → `shared/lib` → `Supabase`.

Capa	Tecnología	Versión	Justificación
Framework UI	React	19.2.4	SPA pura, sin <i>server components</i>
Build/dev server	Vite	8.0.1	Arranque y HMR rápidos
Enrutamiento	TanStack Router	1.168.8	<i>File-based routing</i> tipado, <code>validateSearch</code> con Zod
Estado de servidor	TanStack Query	5.95.2	Caché e invalidación declarativas sobre PostgREST
Formularios	TanStack Form + Zod	1.28.5 / 3.25.76	Validación tipada en el cliente
Estado de cliente	Zustand	5.0.12	Store mínimo (sesión, tema)
Estilos	Tailwind CSS	4.2.2	Tokens CSS-first (<code>@theme</code>)
Componentes UI	shadcn/ui + Radix	—	Primitivos accesibles, código propio
Mapas / geocodificación	Leaflet + Nominatim	1.9.4	Sin costo de licencia de mapas

Capa	Tecnología	Versión	Justificación
BaaS	Supabase (<code>supabase-js</code>)	2.105.1	Ver §11.4
Lenguaje	TypeScript (strict)	5.9.3	Tipado generado desde el esquema real

Tabla 26 — Stack tecnológico por capa, versión y justificación.

❗ Fuente — `frontend/package.json` ; `supabase/config.toml:36` (Postgres `major_version = 17`).

11.4 "Backend" — capa BaaS

Hosty **no tiene un backend a medida**: no existe ningún servicio Node/NestJS en ejecución que reciba peticiones HTTP de la SPA. La capa que cumple ese rol es Supabase, y se compone de tres piezas verificables en `supabase/migrations/*.sql` :

- **Postgres + RLS** como capa de reglas de negocio: cada tabla tiene `enable row level security` y políticas por operación (detalle completo en [Anexo I — Modelo de Datos](#), Tabla 48); las restricciones de dominio (estados válidos, tipos de precio) son `CHECK` constraints, no código de aplicación.
- **PostgREST** como generador automático de API REST sobre el esquema `public` (§11.7).
- **Supabase Auth** para registro, login y sesión (JWT), y **Supabase Storage** para las imágenes de salones (bucket `salon-images` , público en lectura).

El manejo de errores ocurre en el cliente: cada hook de `api/*.queries.ts` / `api/*.mutations.ts` propaga el `error` devuelto por `supabase-js` (que refleja el código de Postgres/PostgREST, incluida una violación de `CHECK` o de política RLS) y TanStack Query lo expone como estado `isError` para que el componente lo muestre.

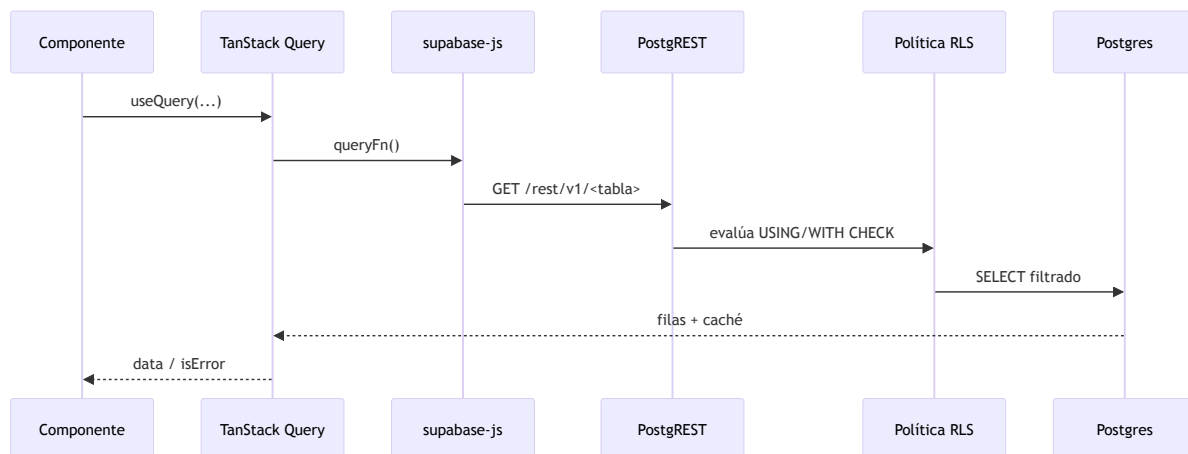


Figura 18 — Ciclo de lectura de datos: componente → hook TanStack Query → `supabase-js` → PostgREST → RLS → Postgres → caché.

11.5 Base de datos

El motor es **Postgres 17** (`supabase/config.toml:36`), sin ORM: los tipos de TypeScript se generan desde el esquema real con `supabase gen types typescript` hacia `frontend/src/shared/lib/database.types.ts`. El esquema `public` tiene 6 tablas (M10) más la tabla `auth.users`, administrada por Supabase Auth. El diagrama entidad-relación completo, el diccionario de datos por tabla y las políticas RLS se documentan en [Anexo I — Modelo de Datos](#) (Figura 28, Tablas 42-49).

11.6 Seguridad

- **Autenticación:** Supabase Auth administra la sesión y el JWT; el frontend nunca implementa hashing de contraseñas propio. El arranque de la sesión al iniciar la aplicación (*bootstrap*) se detalla en la Figura 16 (§11.2).
- **Autorización:** no hay RBAC ni tabla de roles; cada política RLS compara `auth.uid()` contra la columna de propiedad (`host_id` , `user_id`), como se detalla en [Equipo y Roles](#) (Tabla 13) y en el diccionario RLS de [Anexo I — Modelo de Datos](#) (Tabla 48).
- **Validación:** esquemas Zod en el cliente (formularios, `validateSearch` de rutas) más restricciones `CHECK` en Postgres como última línea de defensa, aun si el cliente falla.
- **Sanitización:** todas las consultas usan el *query builder* parametrizado de `supabase-js` sobre PostgREST; no hay concatenación de SQL en ninguna capa del frontend.

Ruta	Control de acceso	Política RLS asociada
<code>/</code> , <code>/salones</code> , <code>/salones/</code> , <code>/salones/\$id</code>	Pública	<code>Salones are publicly readable</code> ; <code>Salon services are publicly readable</code>

Ruta	Control de acceso	Política RLS asociada
/login , /register	Pública	Supabase Auth (sin política de tabla)
/salones/\$id/reservar	requireAuth	Users can insert their own bookings (auth.uid() = user_id)
/mis-reservas	requireAuth	Users can view their own bookings
/mis-favoritos	requireAuth	user_read_own_favorites / user_insert_own_favorites / user_delete_own_favorites
/mi-perfil	requireAuth	Supabase Auth (updateUser)
/host/dashboard , /host/\$bookingId	requireAuth	Hosts can view/update bookings for their salones
/host/create , /host/\$id/edit	requireAuth	Host can insert/update their own salon ; Host manages services/availability of their salones

Tabla 29 — Rutas, control de acceso y política RLS asociada.

11.7 API

No existe una especificación Swagger propia porque no hay un backend a medida: PostgREST expone un documento OpenAPI auto-generado en `{SUPABASE_URL}/rest/v1/` a partir del esquema `public` (detalle de la URL real en [Anexo IV — API y Repositorio](#), y en [Portada y Ficha Técnica](#) como placeholder P-07 de URLs de producción). Cada hook de `api/*.queries.ts` / `*.mutations.ts` es una operación PostgREST; se tabulan por módulo:

Módulo	Consultas	Mutaciones	Total	Tabla(s) principal(es)
salones	5	0	5	salones , salon_availability_blocks
bookings	1	2	3	bookings
favorites	2	1	3	user_favorites
host	6	9	15	salones , bookings , salon_services , salon_availability_blocks , salon_subscriptions , Storage
auth (Supabase Auth, no PostgREST)	—	—	6 (signInWithPassword , signUp , signOut , getSession , onAuthStateChange , updateUser)	auth.users

Módulo	Consultas	Mutaciones	Total	Tabla(s) principal(es)
Total operaciones PostgREST	14	12	26	

Tabla 30 — Operaciones de API por módulo y ambientes de despliegue.

❗ **Fuente** — conteo verificado directamente sobre `frontend/src/features/*/api/*.ts` (2026-07-28).

En cuanto a ambientes: el repositorio sólo define un ambiente de despliegue automatizado, **DEV** (`web-dev.yml` , secretos con sufijo `_DEV`); no existe un workflow de *staging* ni de producción, aunque `infra/config/stg.tfvars` reserva variables para un ambiente `stg` no conectado a ningún pipeline de CI/CD.

11.8 Deployment

El frontend se compila con Vite y se publica en S3, servido por CloudFront (`infra/frontend.tf`), mediante el workflow `web-dev.yml` disparado en cada push a `dev` . Terraform gestiona la infraestructura de forma manual (`terraform plan` vía `infra-ci.yml` , `workflow_dispatch`). Supabase Cloud aloja la base de datos, Auth y Storage; no requiere aprovisionamiento propio. La invalidación de CloudFront tras cada despliegue asegura que los usuarios reciban siempre el build más reciente sin depender del vencimiento natural de la caché del CDN, a costa de una invalidación completa (`/*`) en cada push a `dev` en lugar de una invalidación selectiva por ruta.

Estructura	Responsabilidad
<code>frontend/src/routes/</code>	Definición de rutas, <code>validateSearch</code> , guards
<code>frontend/src/features/<n>/</code>	Componentes, hooks de datos (<code>api/</code>), tipos
<code>frontend/src/shared/lib/</code>	Cliente Supabase, tipos generados, utilidades
<code>frontend/src/components/</code>	<code>layout/</code> (Header, RootLayout) y <code>ui/</code> (shadcn)
<code>infra/</code>	Terraform: S3 + CloudFront (frontend), EC2 + RDS (obsoletos, ver Tabla 27)
<code>supabase/migrations/</code>	Historial de esquema, RLS y datos semilla

Tabla 28 — Estructura de carpetas y responsabilidad.

Decisiones arquitectónicas (ADR resumidas)

#	Decisión	Estado / hallazgo
ADR-1	Migrar de backend NestJS de tres capas a BaaS de dos capas con Supabase	Aplicada en el commit <code>3a89616</code> (2026-04-29); justificada por la escala de un MVP

#	Decisión	Estado / hallazgo
ADR-2	Sin ORM: tipos generados desde el esquema real (<code>database.types.ts</code>)	Vigente; evita drift entre modelo y tipos declarados a mano
ADR-3	Autorización por propiedad vía RLS (<code>auth.uid()</code>), sin tabla de roles/RBAC	Vigente (ver Equipo y Roles , Tabla 13)
Hallazgo A	<code>frontend/package.json</code> declara <code>axios</code> como dependencia de runtime pese a que el proyecto usa exclusivamente <code>supabase-js</code> para acceder a datos	Inconsistencia no resuelta: dependencia sin uso activo identificado en el código de features revisado
Hallazgo B	<code>docker-compose.yml</code> , el <code>package.json</code> raíz (<code>workspaces: ["backend","frontend"]</code>) e <code>infra/backend.tf</code> / <code>infra/rds.tf</code> (EC2 + RDS) siguen describiendo y aprovisionando el backend NestJS eliminado en <code>3a89616</code>	Documentación y definición de infraestructura desactualizadas respecto del código real; no aprovisionadas en este cambio
Hallazgo C	La restricción <code>CHECK</code> de <code>bookings.status</code> no reflejaba los cuatro estados usados por la aplicación hasta la migración <code>20260609233130</code>	Corregido; desarrollado en detalle en Anexo I — Modelo de Datos (Tabla 43) y como deuda técnica en Conclusiones (Tabla 40)
Hallazgo D	La tabla <code>salones</code> careció de política RLS de <code>DELETE</code> hasta la migración <code>20260616000001</code> : con RLS activo y sin esa política, el borrado desde el cliente afectaba 0 filas sin devolver error	Corregido; ver política <code>Host can delete their own salon</code> en Anexo I — Modelo de Datos (Tabla 48)

Tabla 27 — Decisiones arquitectónicas (ADR resumidas).

i Fuente — `commit 3a89616 ( git log )`; `frontend/package.json` ; `docker-compose.yml` ; `package.json` (raíz); `infra/backend.tf` , `infra/rds.tf` ; `supabase/migrations/20260609233130_host_booking_management.sql` ; `supabase/migrations/20260616000001_add_salon_delete_policy.sql` (comentario verbatim del propio archivo). **M22: 0 enums** de Postgres — los dominios de valores válidos se modelan como `CHECK` más uniones de tipo TypeScript mantenidas a mano, el mecanismo que originó el Hallazgo C.

12. Testing y Calidad

Estrategia general

El aseguramiento de calidad no se trata como una etapa posterior al desarrollo sino como una actividad que interviene desde la redacción de las historias de usuario: cada user story destacada en la sección 09 (Planificación Scrum) incluye un criterio de aceptación explícito, y ese criterio es el insumo directo para diseñar los casos de prueba —manuales o automatizados— de la funcionalidad correspondiente. La ejecución de pruebas automatizadas ocurre en dos momentos: localmente, durante el desarrollo, y en la integración continua, al abrirse un *pull request*. Las pruebas manuales y exploratorias se ejecutan antes de cerrar cada historia, sobre el ambiente de desarrollo desplegado.

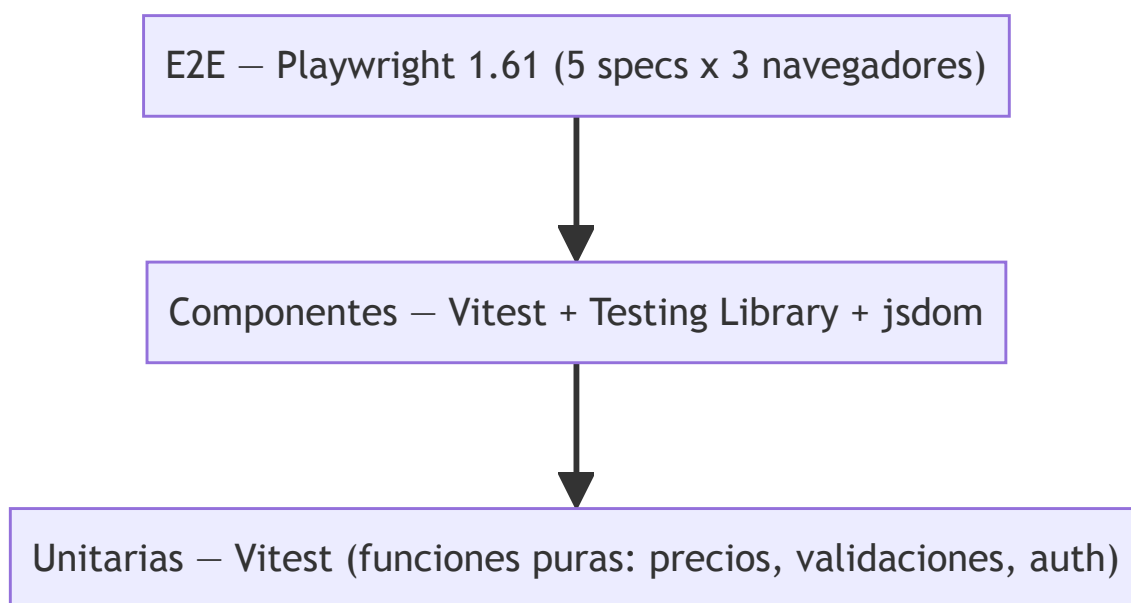


Figura 19 — Pirámide de pruebas: unitarias (Vitest) / componentes (RTL+jsdom) / E2E (Playwright).

i Fuente — M12/M13 (`_meta/Datos-Verificables.md`): 66 pruebas Vitest en 13 archivos unitarios/de componentes, más 5 specs Playwright E2E.

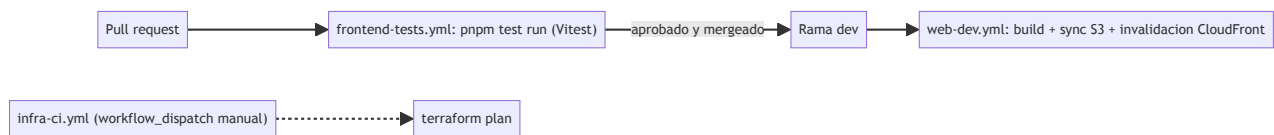


Figura 20 — Pipeline CI/CD: PR → `frontend-tests.yml` (Vitest) → merge a `dev` → `web-dev.yml` (build + sync S3 + invalidación CloudFront); `infra-ci.yml` manual (`terraform plan`).

❗ Fuente — M14 (`_meta/Datos-Verificables.md`): 3 workflows en `.github/workflows/` (`frontend-tests.yml` , `web-dev.yml` , `infra-ci.yml`).

Tipos de prueba

Tipo	Herramienta	Alcance real
Unitarias	Vitest 4.1.6	Funciones puras: <code>pricing.ts</code> (formato de precio), <code>auth.ts</code> (lógica de sesión), validaciones de <code>search-validation.ts</code>
Componentes	Vitest + Testing Library (React) 16.3.2 + jsdom	Renderizado e interacción: <code>CardSalon</code> , <code>Header</code> , <code>LoginPage</code> , <code>RegisterPage</code>
Integración	Vitest, con el cliente de Supabase mockeado	Hooks de <code>api/</code> : <code>bookings.test.ts</code> , <code>favorites.test.ts</code> , <code>salones.queries.test.ts</code> — validan la traducción <code>snake_case</code> → dominio y el manejo de errores de PostgREST
E2E	Playwright 1.61.0, 3 navegadores (Chromium, Firefox, WebKit)	5 specs en <code>src/e2e/</code> : <code>auth-flow</code> , <code>home</code> , <code>navigation</code> , <code>salon-detail</code> , <code>salones</code>
Legacy / exploratorio	Mocha 11.7.6 + Chai 6.2.2 (<code>test:mocha</code>); Cypress 15.17.0	1 archivo Mocha (<code>src/test/mocha/search-validation.test.ts</code>); 1 spec Cypress (<code>cypress/e2e/salon-search.cy.ts</code>)

Tabla 31 — Tipos de prueba, herramienta y alcance real.

❗ Fuente — M12/M13, verificado ejecutando `npx vitest run` sobre el repositorio: 13 archivos, 66 casos, todos en verde. Nota honesta: sólo la suite de Vitest está integrada al pipeline de CI (`frontend-tests.yml` ejecuta `pnpm test run`); Playwright, la corrida independiente de Mocha (`pnpm test:mocha`) y el `spec` de Cypress se ejecutan de forma local o manual y no forman parte de ningún *workflow* de `.github/workflows/` . El archivo de Mocha, además, también es recolectado por Vitest porque su ruta no está excluida en `vite.config.ts` (`exclude: [...configDefaults.exclude, 'src/e2e/**']`); por eso sus 4 casos ya están incluidos en el total de 66.

Cobertura

No hay una herramienta de cobertura de líneas configurada en el proyecto (no existe `--coverage` en el script `test` , ni `@vitest/coverage-v8` / `@vitest/coverage-istanbul` entre las dependencias). Por lo tanto, este informe **no reporta un porcentaje de cobertura de líneas**: hacerlo sin una herramienta que lo mida sería un dato inventado. En su lugar, se reportan únicamente los conteos verificables:

Métrica	Valor
Pruebas automatizadas (Vitest)	66
Archivos de prueba (Vitest/RTL + Playwright)	18 (13 + 5)
Líneas de código de prueba (unitarias + componentes + E2E)	1.462
Líneas de código de producción (<code>src/</code> , sin pruebas)	11.148
Relación líneas de prueba / líneas de producción	≈ 0,13 (13 %)

Tabla 32 — Cobertura de pruebas por módulo.

❗ Fuente — M12/M13; líneas de prueba y de producción contadas con `find frontend/src -name '*.test.ts' -o -name '*.test.tsx' -o -path '*/e2e/*.spec.ts' | xargs wc -l` y su complemento sobre `*.ts` / `*.tsx` , respectivamente (2026-07-28). La cifra de producción incluye `src/routeTree.gen.ts` (343 líneas autogenerated por TanStack Router).

✓ PLACEHOLDER P-46 — Adopción de una herramienta de cobertura de líneas Evaluar e instalar `@vitest/coverage-v8` (u otra) para obtener un porcentaje de cobertura real antes de la próxima entrega; ver también la línea de evolución futura en [Conclusiones](#). Responsable: equipo. Destino: script `test` de `frontend/package.json` .

Matriz de casos de prueba manuales

Se documentan tres casos representativos, mapeados a flujos reales de la aplicación. El campo "Resultado obtenido" no proviene de un registro de ejecución real (no existe un sistema de *test management* en uso), por lo que se marca como dato simulado.

ID	Precondiciones	Pasos	Datos	Resultado esperado	Resultado obtenido
CP-01	Usuario autenticado; salón con un bloqueo de disponibilidad para el 2026-08-10	1. Ir a <code>/salones/:id/reservar</code> . 2. Seleccionar el 2026-08-10 como fecha. 3. Intentar confirmar el paso 1 del wizard	<code>salon_availability_blocks</code> con <code>date = 2026-08-10</code> para el salón	El wizard bloquea el avance y muestra un mensaje de fecha no disponible	(ver SIM-33)
CP-02	Usuario autenticado; salón sin favorito previo	1. Abrir <code>/salones</code> . 2. Click en el ícono de favorito de una <code>CardSalon</code> . 3. Observar el estado del ícono antes de la respuesta del servidor	Salón sin fila en <code>user_favorites</code> para ese usuario	El ícono cambia a "favorito" de inmediato (actualización optimista) y persiste tras recargar	(ver SIM-34)
CP-03	Ninguna (usuario no autenticado)	1. Ir a <code>/login</code> . 2. Ingresar un email válido con una contraseña incorrecta. 3. Enviar el formulario	<code>email: usuario@ejemplo.com</code> , <code>password: incorrecta123</code>	Se muestra un mensaje de error de credenciales inválidas y el usuario permanece en <code>/login</code>	(ver SIM-35)

Tabla 33 — Matriz de casos de prueba manuales.

⚠ **Dato simulado SIM-33 — Resultado obtenido de CP-01** No hay un registro de ejecución manual real para este caso. El resultado se infiere de forma plausible a partir de la prueba de integración equivalente (`bookings.test.ts`) y de la lógica de validación de disponibilidad implementada, pero no debe interpretarse como una ejecución verificada.

⚠ **Dato simulado SIM-34 — Resultado obtenido de CP-02** Ídem SIM-33: se infiere del comportamiento de `useToggleFavorite` (`favorites.test.ts`), que aplica la actualización optimista antes de confirmar la respuesta de Supabase, pero no constituye una ejecución manual registrada.

⚠ **Dato simulado SIM-35 — Resultado obtenido de CP-03** Ídem SIM-33/34: se infiere del test de integración `LoginPage.test.tsx` ("muestra el error del servidor cuando las credenciales son incorrectas"), sin una ejecución manual documentada.

Manejo de incidencias

Las incidencias se reportan como *GitHub Issues* con la etiqueta `bug`. El repositorio registra 13 issues reales con esa etiqueta, las 13 cerradas.

! Fuente — `gh issue list --state all --label bug --json number,state` (2026-07-28): 13 issues, 13 en estado CLOSED .

El ciclo de vida observado es: **Reportado** (se crea el issue) → **Triage** (se prioriza o se etiqueta) → **En curso** (rama `fix/...` asociada) → **En revisión** (*pull request* abierto) → **Retesting** (verificación manual sobre el ambiente DEV tras el *merge*) → **Cerrado**. Una incidencia puede desviarse a **No reproducible** o **Diferida** en la etapa de *triage*.

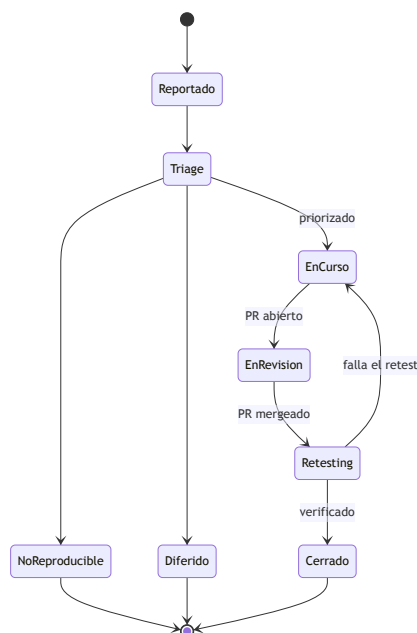


Figura 21 — Ciclo de vida de un defecto: Reportado → Triage → En curso → En revisión → Retesting → Cerrado (+ No reproducible / Diferido).

Un defecto real, no simulado, ilustra este ciclo de punta a punta:

! Fuente — M17: la restricción `CHECK` original de `bookings.status` sólo admitía `pending / confirmed / cancelled` (`supabase/migrations/20240101000000_init_hosty.sql:57-58`), mientras que la aplicación ya emitía un cuarto estado, `declined` . El defecto se resolvió en la migración `supabase/migrations/20260609233130_host_booking_management.sql:7-11` , cuyo propio comentario documenta la causa (" `'declined'` was used by the app but missing from the DB check. "). Ver el detalle completo, con retest, en [Anexo V — Evidencias de QA](#) (Tabla 58) y el análisis de deuda técnica en [Conclusiones](#) (Tabla 40). La severidad de cada incidencia — Bloqueante, Alta, Media o Baja — se clasifica en la Tabla 34 de la siguiente sección, con ejemplos reales tomados de las 13 *issues* bug del repositorio.

Criterios de salida

Una historia se considera lista para cerrar cuando se cumplen, en conjunto: (a) la suite de Vitest pasa en verde localmente y en `frontend-tests.yml` ; (b) `npx eslint .` y `npx tsc -b --noEmit` no reportan errores; (c) el *pull request* asociado cuenta con al menos una aprobación y está mergeado a

`dev` . Estos criterios generales de cierre de historia se completan con una clasificación de severidad de incidencias, que determina además un criterio de salida específico por severidad al cierre de cada sprint.

Severidad	Criterio de calificación en Hosty	Ejemplos reales (issues bug)	Criterio de salida adicional
Bloqueante	Impide completar una reserva o publicar un salón, o corrompe datos, sin ningún <i>workaround</i> disponible	Ninguna de las 13 incidencias reales alcanzó este nivel — banda documentada como vacía, no fabricada	No puede quedar ninguna incidencia Bloqueante abierta al cierre de un sprint; bloquea el <i>merge</i> a <code>dev</code> hasta resolverse
Alta	Una función completa falla o entrega un resultado incorrecto, con o sin <i>workaround</i> manual, fuera del camino crítico de reserva/publicación	p1-high (2): #74 "el mapa en /salones no está implementado — implementar o eliminar"; #75 "agregar a favoritos no persiste — solo estado local"	No puede quedar ninguna incidencia Alta abierta al cierre de un sprint; a lo sumo puede diferirse un (1) sprint con justificación registrada en el backlog
Media	Defecto de navegación, UX o consistencia que degrada la experiencia sin impedir completar el flujo	p2-medium (2): #72 "el link 'Cómo funciona' de la navbar no navega a ninguna sección"; #87 "links del footer apuntan a rutas incorrectas o inexistentes"	Puede diferirse a un sprint posterior si queda registrado en el backlog con responsable asignado
Baja	Defecto cosmético o de bajo impacto, sin efecto funcional sobre ningún flujo	p3-low (1): #76 "links del footer son placeholders — no navegan a destinos reales"	Puede diferirse indefinidamente; no bloquea el cierre de sprint ni el <i>merge</i>

Tabla 34 — Severidad de incidencias y criterios de salida.

i Fuente — `gh issue list --repo juanpablovaldez/hosty --label bug --state all --json number,title,state,labels` (2026-07-28): 13 *issues* con etiqueta `bug` , las 13 cerradas; de ellas, 5 llevan además una etiqueta de prioridad — 2 `p1-high` (#74, #75), 2 `p2-medium` (#72, #87), 1 `p3-low` (#76) — y las 8 restantes no fueron priorizadas explícitamente con esa taxonomía. Verificación en vivo adicional sobre el estado actual del repositorio: `npx tsc -b --noEmit` no reporta errores; `npx eslint .` reporta 6 errores y 4 advertencias (`cypress.config.ts` : parámetros sin usar; `src/test/mocha/search-validation.test.ts` : la regla `no-unused-expressions` no reconoce las aserciones de Chai `expect(...).to.be.true` ; `SalonesPage.tsx` : 4 advertencias de `react-hooks/exhaustive-deps`). El criterio de "lint limpio" no se cumple de forma estricta al momento de esta verificación; se documenta como hallazgo de calidad en [Conclusiones](#) (Tabla 40).

13. Ejecución por Sprint

Esta sección reconstruye la ejecución cronológica del proyecto, sprint por sprint, sobre la base del calendario presentado en [Planificación Scrum](#) (Tabla 21). A diferencia de esa tabla, que resume cantidades, aquí se detallan los entregables concretos de cada sprint, los cambios de alcance o diseño ocurridos durante el desarrollo y la capacidad funcional más distintiva del sistema, ilustrada con un diagrama de secuencia.

i Fuente — Los límites de fecha de cada sprint son una reconstrucción inferida a partir de la densidad de commits y de los clústeres de fecha de las migraciones de Supabase; ver SIM-13 en [Planificación Scrum](#). Los conteos de commits e issues cerradas citados abajo son reales (M23, M24).

Relato por sprint

Sprint	Foco	Entregables	Decisión relevante	Cierre
S1	Arranque, arquitectura base, home	Scaffolding Vite + React 19 + TypeScript, layout base, página de inicio	Elección de stack (React 19, Tailwind v4, Supabase)	2026-04-19
S2	Catálogo, filtros, detalle de salón	Listado dinámico desde Supabase, UI de búsqueda, página de detalle de salón	Migración de NestJS a Supabase (commit <code>3a89616</code>)	2026-05-15
S3	Autenticación, carga de imágenes	Login/registro con Supabase Auth, bucket <code>salon-images</code> , panel y asistente del anfitrión	Adopción de Supabase Storage para imágenes	2026-06-05
S4	Panel del anfitrión, reservas, precios, bloqueos	Drawer de gestión de reservas, precios/servicios flexibles, bloqueos de disponibilidad	Corrección del <code>CHECK</code> de <code>bookings</code> a 4 estados	2026-06-13
S5	Plan destacado, coordenadas, favoritos	Suscripción destacada, geocodificación de salones, favoritos persistentes, política de borrado	Consolidación de 7 pull requests en la PR #93	2026-06-24

Tabla 35 — Sprints: foco, entregables, decisiones y fecha de cierre.



Figura 22 — Commits por mes en la rama dev (marzo-junio 2026).

i Fuente — M01, M03, M04: `git log dev --date=format:'%Y-%m' --format=%ad | sort | uniq -c` (verificado 2026-07-28). Ver [_](#)(ver documentacion-final/meta/Datos-Verificables.md).

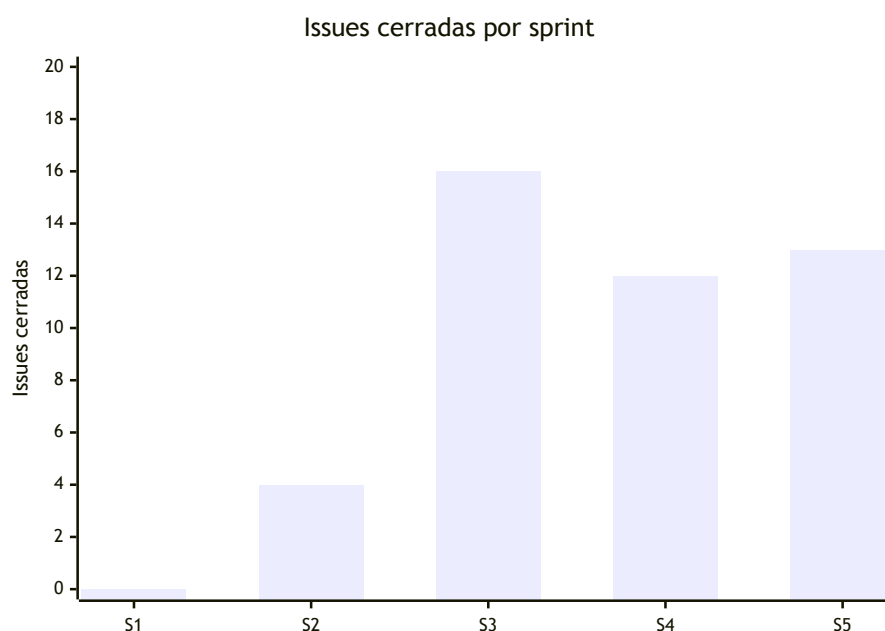


Figura 23 — Issues cerradas por sprint (S1-S5).

i Fuente — M24 (issues cerradas por sprint, calculado para este informe a partir de `closedAt`): `gh issue list --state closed --json number,closedAt` (verificado 2026-07-28). Los 45 cierres suman el total de M06. Ver [_](#)(ver documentacion-final/meta/Datos-Verificables.md).

Cambios de alcance y de diseño

Cambio	Justificación	Evidencia
Migración de NestJS a Supabase	Reducir la complejidad operativa de mantener un backend propio con un equipo part-time; aprovechar autenticación, PostgREST y Storage administrados	Commit <code>3a89616</code> (2026-04-29): remoción completa de <code>backend/</code> y de los workflows de CI/CD asociados
Consolidación de 7 pull requests en una única PR de integración	Evitar conflictos entre ramas de feature abiertas en simultáneo cerca del cierre del proyecto, incluyendo una colisión real de nombre de migración	PR #93, "consolidate all active PRs (#80 #82 #84 #86 #89 #90 #91)", mergeada el 2026-06-23 desde la rama <code>integration/consolidated-prs</code>
Postergación de la integración de Mercado Pago	La pasarela de pago no se completó dentro del período relevado	Issue #45, abierta al cierre del período
Postergación del sistema de reviews y calificaciones	Decisión explícita de alcance, etiquetada <code>post-mvp</code> en el propio backlog	Issue #33, etiqueta <code>post-mvp</code>

Tabla 36 — Cambios de alcance y de diseño con justificación.

¡ Fuente — M26: de las 48 pull requests totales, 20 se cerraron sin fusionar (41,7 %) — `gh pr list --state all --json number,state,mergedAt` (verificado 2026-07-28). Siete de esas 20 corresponden directamente al evento de consolidación descrito arriba (#80, #82, #84, #86, #89, #90, #91); el resto responde a un patrón similar de ramas reintegradas por otra vía a lo largo del proyecto, sin que exista un registro explícito del motivo caso por caso. Ver `_`(ver documentacion-final/meta/Datos-Verificables.md).

Capacidad más distintiva: cotización y confirmación de una reserva

La funcionalidad que mejor distingue a Hosty de un simple formulario de contacto es el flujo de cotización y confirmación de reservas del anfitrión, que combina un precio ajustable (`quotedPrice`) con el bloqueo de fechas (`salon_availability_blocks`).

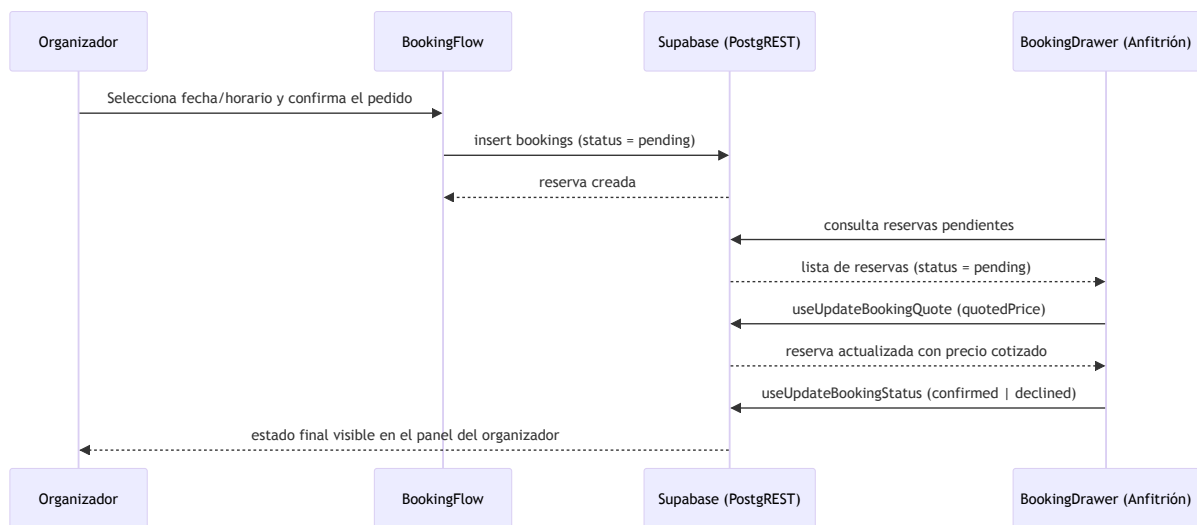


Figura 24 — Capacidad más distintiva, extremo a extremo: huésped reserva → anfitrión cotiza (`quotedPrice`) → confirma/rechaza → estado final del huésped.

¡ Fuente — `frontend/src/features/bookings/components/BookingFlow.tsx` ,
`frontend/src/features/host/components/BookingDrawer.tsx` ,
`frontend/src/features/host/api/host.mutations.ts` (`useUpdateBookingQuote` ,
`useUpdateBookingStatus`),
`supabase/migrations/20260610082550_salon_availability_blocks.sql` (verificado 2026-07-28).

14. Métricas

Todas las cifras de esta sección se toman de `_`(ver `documentacion-final/meta/Datos-Verificables.md`) (`M01` – `M22`), fuente única del vault, y no se repiten sin su identificador `M##` .

Métricas de repositorio y de gestión

Métrica	Valor	Fuente
Duración del proyecto	88 días (≈12,6 semanas), 2026-03-29 a 2026-06-24	M03, M04
Commits en <code>dev</code>	181	M01
Commits en todas las refs	236	M02
Contribuidores	5 (9 identidades Git)	M05
Issues totales / cerradas	50 / 45 (90 %)	M06
Pull requests totales / mergeados	48 / 26	M07
Milestones (épicas)	7	M08
Sprints reconstruidos	≈5–6 (S1–S5, ver Ejecución por Sprint)	Clústeres de <code>supabase/migrations/*.sql</code>
User stories destacadas	≈15 principales, sobre un backlog de 50 issues	Anexo III (Backlog de User Stories)
Ambientes desplegados	1 (DEV)	<code>web-dev.yml</code> (único <i>workflow</i> de despliegue; sin <code>web-staging.yml</code> ni <code>web-prod.yml</code>)

Tabla 37 — Métricas de repositorio y de gestión.

i Fuente — **M01–M08, M14**; ambiente único verificado listando `.github/workflows/` (`frontend-tests.yml` , `web-dev.yml` , `infra-ci.yml` : ninguno despliega a `staging` ni `prod`), 2026-07-28.

Commits por contribuidor (todas las refs, total 236 = M02)

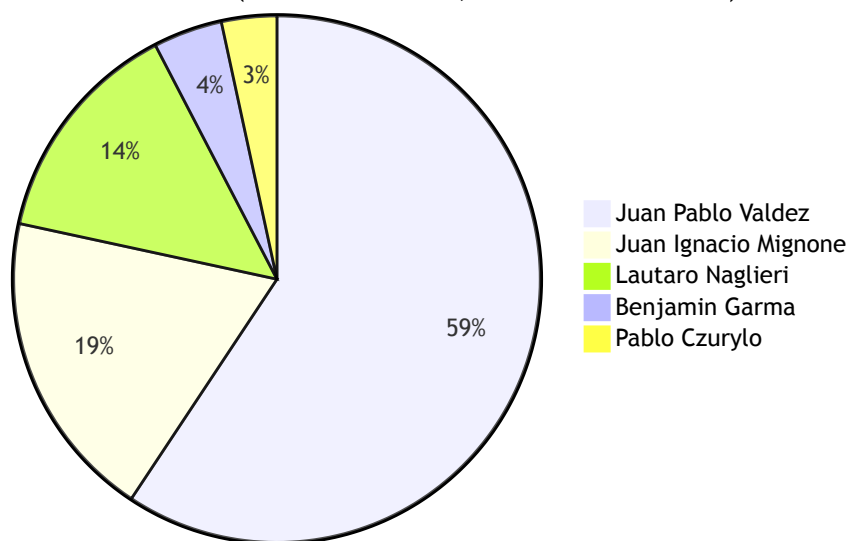


Figura 25 — Distribución de commits por contribuidor (5 contribuidores).

¡ Fuente — M05: `git shortlog -sne --all` (2026-07-28), identidades consolidadas por email en [_ \(ver documentacion-final/meta/Datos-Verificables.md\)](#). Un mismo contribuidor puede tener más de una identidad Git (por ejemplo, dos direcciones distintas para Juan Pablo Valdez); la consolidación agrupa por persona, no por email.

Métricas de producto y de calidad

Métrica	Valor	Fuente
Entidades del modelo de datos	6 tablas públicas	M10
Archivos de migración	10	M11
Rutas / protegidas	14 / 8	M09
Features del frontend	8 módulos	M15
Operaciones PostgREST (invocaciones <code>select / insert / update / delete</code> en <code>api/*.ts</code>)	35, repartidas en 4 módulos activos (ver Anexo IV, API y Repositorio, Tabla 53)	Conteo propio, <code>grep</code> sobre <code>frontend/src/features/*api/*.ts</code>
Pruebas automatizadas por tipo	66 Vitest (13 archivos) + 5 specs Playwright E2E (× 3 navegadores) + 1 Mocha + 1 Cypress locales	M12, M13
Workflows de CI/CD	3	M14

Tabla 38 — Métricas de producto y de calidad.

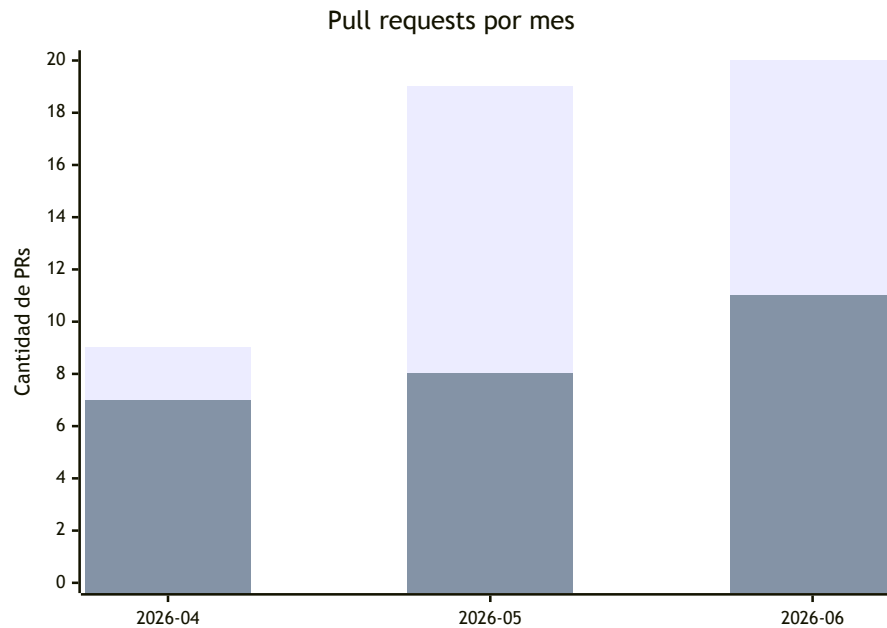


Figura 26 — Pull requests abiertos vs. mergeados por mes.

¡ Fuente — M07: `gh pr list --state all --json number,createdAt,mergedAt` (2026-07-28), agrupado por mes de creación y de *merge*. Total: 48 abiertos, 26 mergeados.

Interpretación: el volumen de *pull requests* abiertos crece mes a mes ($9 \rightarrow 19 \rightarrow 20$), pero la proporción mergeada por mes cae de forma relativa (78 % en abril, 42 % en mayo, 55 % en junio), consistente con la caída de commits observada en junio en [Ejecución por Sprint](#) (Figura 22): hacia el cierre del proyecto se concentró más trabajo en *pull requests* de integración y consolidación (ramas como `integration/consolidated-prs`), que tardan más en revisarse y mergearse que los cambios incrementales de abril y mayo.

15. Conclusiones

Balance funcional

Del backlog total de 50 issues, 45 se cerraron (90 %) y 5 quedaron diferidos, cada uno con una justificación explícita registrada en GitHub.

Issue	Título	Estado	Justificación del diferimiento
#45	feat(payments) : integrar Mercado Pago para reservas	Diferido	Requiere una cuenta comercial y credenciales de producción fuera del alcance del MVP académico
#38	chore(design) : documentar todos los color tokens del brandbook	Diferido	Tarea de documentación de diseño sin impacto funcional; no bloquea ninguna épica
#35	perf : auditar y mejorar Core Web Vitals	Diferido	Optimización de performance planificada como mejora post-entrega, no como requisito del MVP
#33	feat(social) : implementar sistema de reviews y ratings	Diferido, etiquetado post-mvp	Declarado explícitamente fuera del alcance del MVP en su propia etiqueta
#23	DOCS-01 : Final Project Report & Handoff	En curso (es el propio cambio que produce este vault)	Se resuelve con la creación de documentacion-final/

Tabla 39 — Balance funcional: planificado vs. entregado.

i Fuente — `gh issue list --state all --json number,state --limit 300 : 50 totales, 45 CLOSED (2026-07-28)`. Detalle de los 5 diferidos: `gh issue list --json number,title,state,labels` filtrado por número.

Las 7 épicas planificadas (E1–E7, ver [Objetivos](#) y la sección 09, Planificación Scrum) alcanzaron estado funcional en el ambiente de DEV: catálogo y búsqueda, autenticación, reserva, panel del anfitrión, favoritos y plan destacado, calidad e integración continua, e infraestructura y despliegue.

Balance técnico y metodológico

Adoptar Supabase como *backend as a service* permitió al equipo entregar autenticación, control de acceso y persistencia sin escribir ni operar un servidor propio: la autorización se resuelve íntegramente con políticas RLS sobre `auth.uid()` (ver la sección 11, Arquitectura), lo que eliminó una capa completa de código (controladores, DTOs, guards) que un backend propio en NestJS hubiera requerido escribir y mantener. El costo de esa decisión es la dependencia total del modelo de permisos de Postgres: cualquier regla de negocio que no se exprese como una política RLS queda sin protección a nivel de datos.

La gestión con Scrum real —milestones de GitHub mapeados 1 a 1 con las épicas (M08), un tablero de GitHub Projects v2 (M19) y *pull requests* vinculados a issues— dejó un rastro verificable de todo el proceso: los 181 commits de `dev`, los 48 *pull requests* y los 50 issues son, en conjunto, la evidencia primaria de este informe, no una reconstrucción posterior.

Deuda técnica

Hallazgo	Severidad	Impacto	Remediación propuesta
Deriva entre el <code>CHECK</code> de <code>bookings.status</code> en Postgres y el <i>union type</i> de TypeScript (ver detalle abajo)	Media	El estado <code>declined</code> fue usado por la aplicación antes de ser aceptado por la base; el drift se corrigió, pero nada impide que se repita	Generar los tipos de dominio de estado desde la base de datos (o migrar a un <code>enum</code> de Postgres) en vez de mantenerlos a mano en TS
0 <code>enum</code> de Postgres en todo el esquema (M22); todos los dominios cerrados se implementan como <code>CHECK</code> + <code>text</code>	Media	Mayor superficie para que un valor nuevo en la aplicación no tenga correlato en la restricción de base	Evaluar <code>CREATE TYPE ... AS ENUM</code> para <code>bookings.status</code> y <code>salones</code> tipo de precio
<code>axios</code> declarado como dependencia de <code>frontend/package.json</code>	Media	Contradice la arquitectura BaaS documentada (CLAUDE.md prohíbe <code>axios</code> en <code>frontend/</code> ; todo acceso a datos debe ir por <code>supabase-js</code>)	Auditar si <code>axios</code> se usa realmente; si no, quitarlo del <code>package.json</code>
Documentación previa desactualizada: <code>README.md</code> y <code>docs/tech-stack.md</code> describen un backend NestJS eliminado del repositorio, y <code>docs/tech-stack.md</code> todavía llama al proyecto "SalonSpot"	Media	Un lector nuevo del repositorio recibe información arquitectónica falsa	Reescribir <code>docs/tech-stack.md</code> para reflejar la arquitectura Supabase/BaaS actual (fuera del alcance de este cambio, ver Exclusiones de Alcance)
Inconsistencia de gestor de paquetes: <code>frontend/</code> y la raíz tienen tanto <code>package-lock.json</code> como <code>pnpm-lock.yaml</code> ; CLAUDE.md indica usar <code>npm</code> en <code>frontend/</code> , pero <code>frontend-tests.yml</code> y <code>web-dev.yml</code> instalan con <code>pnpm</code>	Media	Riesgo de que las dependencias instaladas localmente (npm) diverjan de las de CI (pnpm)	Fijar un único gestor de paquetes para todo el monorepo y eliminar el lockfile sobrante
<code>root package.json</code> aún declara el workspace <code>backend</code> y scripts <code>docker:* / lint-staged</code> sobre <code>backend/src/**</code> , pese a que el directorio <code>backend/</code> fue eliminado del disco	Baja	Scripts inertes, potencial confusión sobre si el backend NestJS sigue vigente	Quitar <code>backend</code> de <code>workspaces</code> y los scripts asociados

Hallazgo	Severidad	Impacto	Remediación propuesta
Sólo la suite de Vitest corre en CI (<code>frontend-tests.yml</code>); Playwright, Mocha y Cypress se ejecutan únicamente en local	Baja	Regresiones E2E o de los specs legacy pueden llegar a <code>dev</code> sin detectarse automáticamente	Agregar un job de Playwright a CI (o a un <i>workflow</i> nocturno)
<code>tsconfig.app.json</code> excluye <code>src/test</code> , <code>*.test.ts(x)</code> y <code>*.spec.ts(x)</code> del <i>type-check</i> de build	Baja	Errores de tipos dentro de los propios tests no bloquean <code>npm run build</code>	Crear un <code>tsconfig.test.json</code> referenciado que sí tipe los archivos de prueba
<code>prettier</code> está scripteado (<code>format</code> , <code>format:check</code>) pero no figura como dependencia directa de <code>frontend/package.json</code> ; sólo está presente de forma transitiva en <code>node_modules</code>	Baja	El script puede romperse si la dependencia transitiva que lo provee cambia	Declarar <code>prettier</code> como <code>devDependency</code> explícita
<code>react-i18next</code> está inicializado (<code>src/i18n/</code>) pero no se usa en ningún componente (<code>grep -rI useTranslation frontend/src</code> no devuelve resultados)	Baja	Infraestructura de internacionalización sin efecto — todo el texto sigue <i>hardcodeado</i> en español	Adoptar <code>useTranslation</code> de forma incremental o quitar la dependencia si no se usará
No hay herramienta de cobertura de líneas configurada (ver Testing y Calidad)	Baja	No es posible verificar objetivamente qué proporción del código está probada	Instalar <code>@vitest/coverage-v8</code>
<code>npx eslint .</code> reporta 6 errores y 4 advertencias sobre el estado actual del repositorio (ver Testing y Calidad , Tabla 34)	Baja	El criterio de salida "lint limpio" no se cumple de forma estricta hoy	Corregir los parámetros sin usar de <code>cypress.config.ts</code> , ajustar la regla <code>no-unused-expressions</code> para aserciones de Chai, y resolver las dependencias de <code>useMemo</code> en <code>SalonesPage.tsx</code>

Tabla 40 — Deuda técnica: severidad, impacto y plan de remediación.

i Fuente — M22 (`_meta/Datos-Verificables.md`): 0 resultados para `grep -rn "CREATE TYPE\|ENUM" supabase/migrations/*.sql` (2026-07-28). El hallazgo del **CHECK** de `bookings.status` cita `supabase/migrations/20240101000000_init_hosty.sql:57-58` (restricción original de 3 valores) y `supabase/migrations/20260609233130_host_booking_management.sql:7-11` (comentario verbatim: *"declined" was used by the app but missing from the DB check.*), además del *union type* de 4 estados en `frontend/src/features/host/lib/booking-status.ts`. Detalle completo y retest en el Anexo V (Evidencias de QA, Tabla 58) y en [Testing y Calidad](#).

Aprendizajes y líneas de evolución futura

⚠ **Dato simulado SIM-36 — Aprendizajes del equipo** No existe un registro documental de retrospectivas individuales que permita citar textualmente qué aprendió cada integrante. De forma plausible, a partir de la naturaleza del proyecto —una plataforma construida sobre un *backend as a service*, gestionada con Scrum real sobre GitHub—, se reconstruyen dos aprendizajes verosímiles: (a) el equipo profundizó en el modelado de autorización con Postgres RLS como alternativa a un *backend* propio, incluyendo sus límites (una regla de negocio no expresable como política RLS requiere lógica adicional en el cliente o funciones de base de datos); (b) la gestión de alcance con *milestones* e issues etiquetados (`post-mvp` , `bug` , `feature`) ayudó a sostener 45 de 50 issues entregados sin perder trazabilidad sobre lo diferido. Ninguna de estas dos afirmaciones debe tomarse como cita textual de una retrospectiva real.



Figura 27 — Roadmap de evolución: corto (deuda técnica) / medio (i18n, pagos) / largo (multi-provincia).

Horizonte	Línea de evolución	Relación con un hallazgo verificado
Corto plazo	Resolver la deuda técnica de la Tabla 40	Deriva de <code>bookings.status</code> , dependencias duplicadas, cobertura ausente
Mediano plazo	Integrar Mercado Pago	Issue diferido #45
Mediano plazo	Activar <code>react-i18next</code> (<code>useTranslation</code>)	<code>src/i18n/</code> inicializado sin uso (Tabla 40)
Mediano plazo	Ambientes <code>staging</code> y <code>prod</code>	Sólo <code>web-dev.yml</code> despliega hoy (Tabla 37, sección 14)
Largo plazo	Reviews y ratings, panel de administración	Issue diferido #33 (<code>post-mvp</code>)
Largo plazo	Expansión multi-provincia	Extensión natural del catálogo geolocalizado (sección 08)

Tabla 41 — Aprendizajes y líneas de evolución futura.

Anexo I. Modelo de Datos

El esquema `public` tiene 6 tablas (M10) más `auth.users`, administrada por Supabase Auth. No se usa ningún ORM: los tipos de TypeScript se generan directamente desde el esquema real (`frontend/src/shared/lib/database.types.ts`). Este anexo documenta el diagrama entidad-relación completo, el diccionario de datos por tabla, las políticas RLS y el historial de migraciones; la discusión arquitectónica de este modelo está en [Arquitectura](#), y las métricas M10/M11/M17/M22 citadas a lo largo del anexo se consolidan, con su comando de reproducción, en `_(ver documentacion-final/meta/Datos-Verificables.md)`.

Diagrama entidad-relación

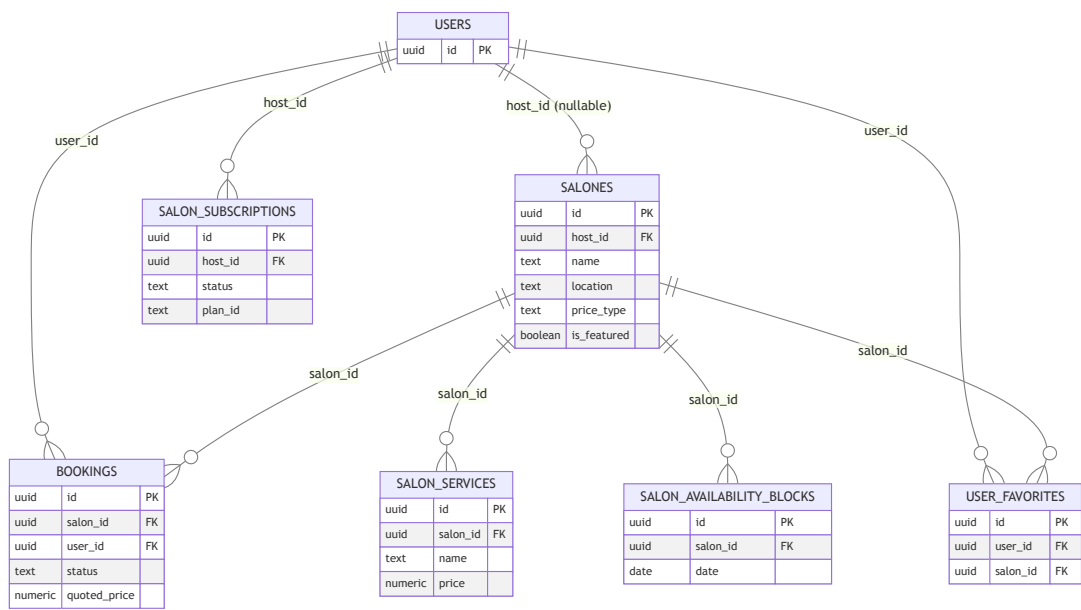


Figura 28 — Modelo de datos completo: `auth.users` + las 6 tablas públicas con cardinalidades y claves foráneas.

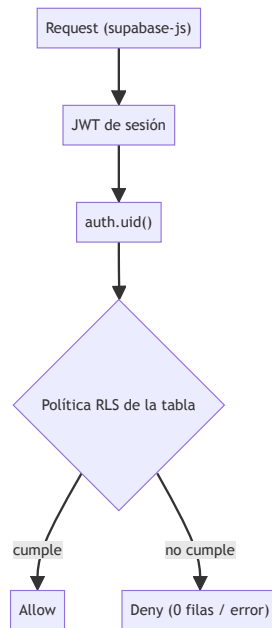


Figura 29 — Cadena de autorización por propiedad: request → JWT → `auth.uid()` → política RLS → allow/deny.

Las ocho relaciones del diagrama son todas de cardinalidad uno a muchos desde `auth.users` o desde `salones` hacia las tablas dependientes; ninguna tabla del esquema tiene una relación muchos a muchos directa. `salones.host_id` es la única clave foránea nullable de todo el modelo (`on delete set null`): un salón puede quedar sin propietario si la cuenta del anfitrión se elimina, en lugar de eliminarse en cascada junto con ella. Todas las demás relaciones hacia `salones` (`bookings` , `salon_services` , `salon_availability_blocks` , `user_favorites`) se borran en cascada cuando se elimina el salón.

Diccionario de datos

La tabla `salones` es la entidad central del modelo: concentra tanto los datos de catálogo (nombre, ubicación, capacidad, imágenes) como el estado comercial del anfitrión (modo de precio, plan destacado, verificación editorial). El campo `price_type` determina qué otras columnas de precio son relevantes: `price_per_hour` sólo se usa cuando el precio es fijo, y `price_min` / `price_max` sólo cuando es un rango estimado; con `on_request` ninguno de los tres se completa y el anfitrión cotiza manualmente desde el panel (ver [Anexo II — Diagramas de Flujo Complementarios](#)).

Columna	Tipo SQL	Restricción	Descripción
<code>id</code>	<code>uuid</code>	PK, default <code>gen_random_uuid()</code>	Identificador
<code>created_at</code>	<code>timestampz</code>	<code>not null default now()</code>	Alta del registro
<code>name</code>	<code>text</code>	<code>not null</code>	Nombre del salón
<code>description</code>	<code>text</code>	<code>nullable</code>	Descripción
<code>location</code>	<code>text</code>	<code>not null</code>	Zona (Tucumán)
<code>address</code>	<code>text</code>	<code>not null</code>	Dirección completa

Columna	Tipo SQL	Restricción	Descripción
price_per_hour	numeric(12,2)	nullable	Precio fijo por hora (sólo price_type='fixed')
price_type	text	check in ('fixed','estimated','on_request')	Modo de precio (M18)
price_min / price_max	numeric(12,2)	nullable	Rango estimado
capacity	int	not null	Capacidad máxima
rating_value / rating_count	numeric(3,2) / int	nullable	Calificación agregada
is_verified	boolean	not null default false	Verificación editorial
is_featured	boolean	not null default false	Plan Destacado activo
availability_status	text	check in ('disponible','reservado','no disponible')	Estado operativo
event_types / amenities / images	text[]	not null default '{}'	Listas
host_id	uuid	FK → auth.users(id) on delete set null , nullable	Propietario
rent_time_hours	int	not null default 1	Alquiler mínimo en horas
latitude / longitude	double precision	nullable	Geolocalización

Tabla 42 — Diccionario de datos — salones.

Columna	Tipo SQL	Restricción	Descripción
id	uuid	PK	Identificador
salon_id	uuid	not null FK → salones(id) on delete cascade	Salón reservado
user_id	uuid	not null FK → auth.users(id) on delete cascade	Huésped
event_date , start_time , end_time	date , time , time	not null	Fecha y horario
attendees	int	not null	Asistentes
event_type	text	not null	Tipo de evento

Columna	Tipo SQL	Restricción	Descripción
status	text	check in ('pending', 'confirmed', 'declined', 'cancelled')	Estado de la reserva — ver hallazgo abajo (M17)
total_price / quoted_price	numeric(12,2)	nullable	Precio final / cotizado por el anfitrión
selected_services	jsonb	not null default '[]'	Servicios extra elegidos
rejection_reason , contact_name , contact_phone	text	nullable	Datos agregados por gestión del anfitrión

Tabla 43 — Diccionario de datos — bookings.

La tabla `bookings` registra tanto la solicitud original del huésped (fecha, horario, asistentes, servicios elegidos) como el resultado de la gestión del anfitrión (`quoted_price` , `rejection_reason`). El wizard de reserva de tres pasos descrito en [Anexo II — Diagramas de Flujo Complementarios](#) escribe una única fila con `status = 'pending'` ; las transiciones posteriores las produce el panel del anfitrión mediante actualizaciones parciales sobre esa misma fila, nunca filas nuevas.

¡ Fuente — M17: hallazgo verificado sobre status . La migración inicial `supabase/migrations/20240101000000_init_hosty.sql:57-58` sólo permitía tres valores (`'pending'` , `'confirmed'` , `'cancelled'`). El comentario verbatim de `supabase/migrations/20260609233130_host_booking_management.sql:7-11` documenta el defecto: *"declined" was used by the app but missing from the DB check.* La migración reemplaza la restricción por `check (status in ('pending', 'confirmed', 'declined', 'cancelled'))` . El tipo TypeScript de `frontend/src/features/host/lib/booking-status.ts` ya modelaba las cuatro variantes antes de que la base de datos las aceptara: drift real entre el CHECK de Postgres y el dominio TS, sin ningún enum de Postgres de por medio (M22 = 0). Desarrollado como deuda técnica en [Conclusiones](#) (Tabla 40) y en [Arquitectura](#) (Tabla 27, Hallazgo C).

Columna	Tipo SQL	Restricción	Descripción
id	uuid	PK	Identificador
salon_id	uuid	not null FK → salones(id) on delete cascade	Salón
name	text	not null	Nombre del servicio
price	numeric(12,2)	nullable ("a consultar" si es null)	Precio del servicio

Tabla 44 — Diccionario de datos — salon_services.

Columna	Tipo SQL	Restricción	Descripción
id	uuid	PK	Identificador
salon_id	uuid	not null FK → salones(id) on delete cascade	Salón bloqueado

Columna	Tipo SQL	Restricción	Descripción
date	date	not null , unique(salon_id, date)	Día no disponible
reason	text	nullable	Motivo

Tabla 45 — Diccionario de datos — salon_availability_blocks.

salon_services y salon_availability_blocks son ambas sub-recursos de salones administrados exclusivamente por el anfitrión propietario (política RLS ALL), pero de lectura pública: el huésped las consulta durante el flujo de reserva sin necesitar sesión iniciada, ya que la disponibilidad y los servicios extra de un salón son información de catálogo, no privada.

Columna	Tipo SQL	Restricción	Descripción
id	uuid	PK	Identificador
user_id	uuid	not null FK → auth.users(id)	Usuario
salon_id	uuid	not null FK → salones(id) on delete cascade	Salón favorito
created_at	timestampz	not null default now() , unique(user_id, salon_id)	Alta

Tabla 46 — Diccionario de datos — user_favorites.

La restricción unique(user_id, salon_id) es la única regla de integridad que impide un favorito duplicado; la lógica de alternar (agregar/quitar) vive enteramente en el cliente, como se detalla en [Anexo II — Diagramas de Flujo Complementarios](#) (Figura 34).

Columna	Tipo SQL	Restricción	Descripción
id	uuid	PK	Identificador
host_id	uuid	not null FK → auth.users(id)	Anfitrión suscriptor
status	text	check in ('pending', 'active', 'cancelled', 'expired')	Estado de la suscripción
plan_id	text	not null default 'destacado'	Plan contratado
amount_monthly	numeric(10,2)	not null default 4999	Monto mensual
mercadopago_subscription_id	text	nullable	Referencia externa de pago
started_at , current_period_end , cancelled_at	timestampz	nullable	Ciclo de vida

Tabla 47 — Diccionario de datos — salon_subscriptions.

`salon_subscriptions` no tiene relación directa con `salones` a nivel de clave foránea: el vínculo comercial se cierra mediante la aplicación, que al cancelar una suscripción actualiza por separado `salones.is_featured` a `false` para el `host_id` correspondiente. El campo `mercadopago_subscription_id` anticipa una integración de cobro que, a la fecha de esta verificación, no está conectada a un flujo de pago real.

Políticas RLS por tabla y operación

Todas las tablas del esquema `public` tienen Row Level Security habilitada; no existe ninguna tabla de negocio con lectura o escritura sin restricción. El patrón dominante es "lectura pública, escritura por propietario": el catálogo (`salones` , `salon_services` , `salon_availability_blocks`) es visible sin sesión, mientras que las tablas de datos personales (`bookings` , `user_favorites` , `salon_subscriptions`) exigen coincidencia de `auth.uid()` incluso para `SELECT` .

Tabla	Operación	Regla
<code>salones</code>	SELECT	Pública (<code>using (true)</code>)
<code>salones</code>	INSERT / UPDATE / DELETE	Propietario (<code>auth.uid() = host_id</code>)
<code>bookings</code>	SELECT	Huésped propio (<code>user_id</code>) o anfitrión del salón (<code>salon_id in (... host_id = auth.uid())</code>)
<code>bookings</code>	INSERT	Huésped propio (<code>auth.uid() = user_id</code>)
<code>bookings</code>	UPDATE	Huésped propio (cancelar) o anfitrión del salón (confirmar/rechazar/cotizar)
<code>salon_services</code>	SELECT	Pública
<code>salon_services</code>	ALL (host)	Anfitrión dueño del salón relacionado
<code>salon_availability_blocks</code>	SELECT	Pública
<code>salon_availability_blocks</code>	ALL (host)	Anfitrión dueño del salón relacionado
<code>user_favorites</code>	SELECT / INSERT / DELETE	Propietario (<code>user_id = auth.uid()</code>)
<code>salon_subscriptions</code>	SELECT / INSERT / UPDATE	Propietario (<code>host_id = auth.uid()</code>)
<code>storage.objects</code> (<code>salon-images</code>)	INSERT	Cualquier usuario autenticado
<code>storage.objects</code> (<code>salon-images</code>)	SELECT	Pública
<code>storage.objects</code> (<code>salon-images</code>)	UPDATE / DELETE	Dueño de la ruta (<code>salones/{auth.uid()}/...</code>)

Tabla 48 — Políticas RLS por tabla y operación.

i Fuente — hallazgo verificado adicional: la tabla `salones` no tuvo política de `DELETE` hasta `supabase/migrations/20260616000001_add_salon_delete_policy.sql`. El comentario verbatim del archivo documenta el efecto: con RLS activo y sin política de `DELETE`, el borrado desde el cliente afectaba 0 filas sin devolver error, de modo que el salón nunca se eliminaba. Corregido con la política `Host can delete their own salon`. Ver también [Arquitectura](#) (Tabla 27, Hallazgo D).

Historial de migraciones

El esquema creció de forma incremental a lo largo de los cinco sprints documentados en [Ejecución por Sprint](#): la migración inicial cubre sólo `salones` y `bookings`; el resto de las tablas y columnas se agregó a medida que se incorporaron precios flexibles, gestión de disponibilidad, plan destacado, coordenadas geográficas y favoritos.

Migración	Contenido
<code>20240101000000_init_hosty</code>	Esquema inicial: <code>salones</code> , <code>bookings</code> , RLS base, datos semilla
<code>20260516000001_add_performance_indexes</code>	Índices GIN/ <code>trgm</code> y de rango para búsqueda
<code>20260525000001_create_storage_bucket</code>	Bucket <code>salon-images</code> (M16) y políticas de Storage
<code>20260609233130_host_booking_management</code>	Estado <code>declined</code> , motivo de rechazo, RLS de anfitrión sobre <code>bookings</code>
<code>20260609234240_pricing_and_services</code>	Precios flexibles, <code>salon_services</code> , <code>quoted_price</code>
<code>20260610082550_salon_availability_blocks</code>	Bloqueos de disponibilidad por fecha
<code>20260614000001_host_destacado_plan</code>	<code>salon_subscriptions</code> , <code>is_featured</code>
<code>20260614000002_add_salon_coordinates</code>	<code>latitude</code> / <code>longitude</code> y backfill de semillas
<code>20260616000001_add_salon_delete_policy</code>	Política de <code>DELETE</code> faltante en <code>salones</code>
<code>20260617000001_user_favorites</code>	Tabla <code>user_favorites</code>

Tabla 49 — Historial de migraciones.

i Fuente — M11: 10 archivos de migración (`supabase/migrations/*.sql`). La marca de tiempo `20240101000000` del archivo inicial es un valor placeholder anterior a la creación real del repositorio (2026-03-29, M03) y no debe leerse como fecha real de ese cambio.

Anexo II. Diagramas de Flujo Complementarios

Este anexo detalla los flujos de proceso que [Diseño y Desarrollo](#) referencia sin diagramar, para mantener esa sección centrada en la interfaz. La lectura de estos flujos complementa el modelo de datos de [Anexo I — Modelo de Datos](#) y la arquitectura de [Arquitectura](#).

Búsqueda y filtrado

La búsqueda no requiere sesión iniciada: los filtros de la ruta `/salones/` se codifican enteramente en la URL (`validateSearch` con Zod), por lo que un resultado de búsqueda es enlazable y compatible. El ordenamiento y la paginación se resuelven del lado del servidor (PostgREST), no en el cliente, para no descargar el catálogo completo en cada búsqueda.

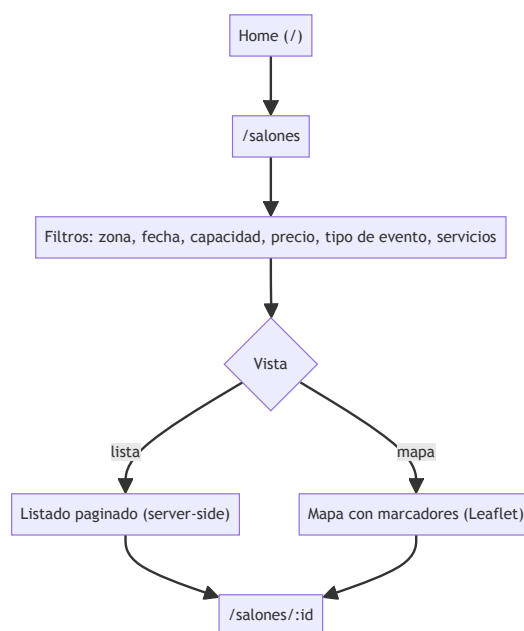


Figura 30 — Flujo de búsqueda y filtrado: `home` → `/salones` → `filtros/mapa` → `/salones/:id`.

¡ Fuente — la ruta `/salones/` valida 17 parámetros de búsqueda con Zod
(`frontend/src/routes/salones/index.tsx` , `searchSchema` ; verificado 2026-07-28).

Flujo de reserva

El asistente exige sesión iniciada (`requireAuth`) recién al llegar a `/salones/:id/reservar` ; los dos primeros pasos se validan íntegramente en el cliente antes de escribir nada en la base de datos, y sólo la confirmación del paso 3 dispara la mutación. Si el salón cotiza "a consultar", el total se muestra como pendiente de cotización en lugar de un monto, y la reserva igual se crea en estado `pending` .

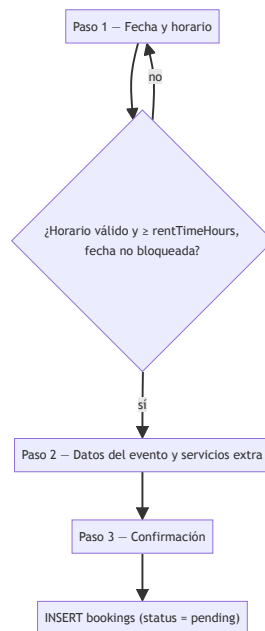


Figura 31 — Flujo de reserva (wizard de 3 pasos): fecha y horario → datos del evento → confirmación.

¡ Fuente — M20: 3 pasos (frontend/src/features/bookings/components/BookingFlow.tsx).
Valida horario mínimo (salon.rentTimeHours) y fechas de salon_availability_blocks .

Publicación de un salón (anfitrión)

El mismo componente (SalonWizard) se reutiliza en modo creación y en modo edición: en edición, el paso de imágenes conserva las URLs existentes y sólo sube a Storage los archivos nuevos, y el paso final reemplaza el registro completo de salones en lugar de aplicar un parche parcial.

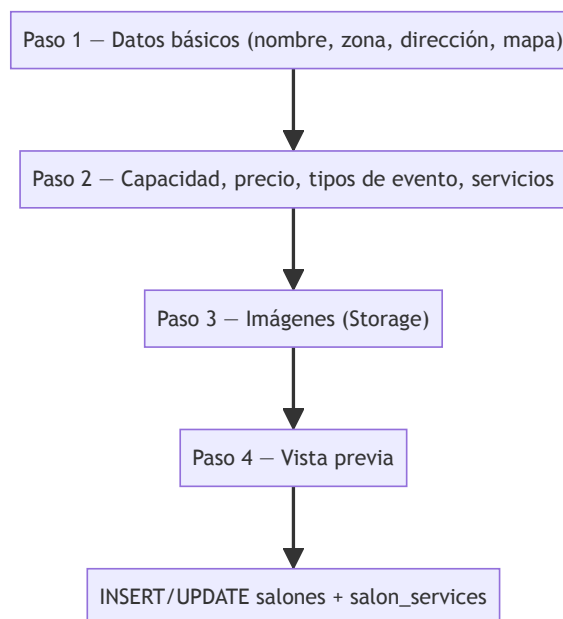


Figura 32 — Flujo de publicación de salón (wizard de 4 pasos): datos básicos → capacidad/precio/servicios → imágenes → vista previa.

❗ Fuente — M21: 4 pasos (`frontend/src/features/host/components/SalonWizard.tsx`).

Máquina de estados de una reserva

Toda transición la ejecuta el anfitrión desde su panel, salvo la cancelación, que también puede iniciarla el huésped desde `/mis-reservas`. No existe una transición automática por vencimiento de fecha: una reserva `confirmed` para un evento ya pasado permanece en ese estado indefinidamente si nadie la actualiza manualmente.

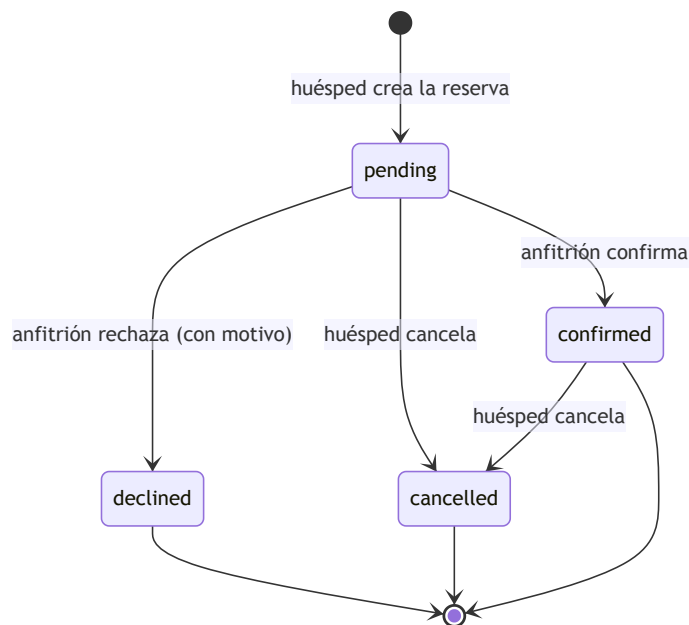


Figura 33 — Máquina de estados de una reserva: `pending` → `confirmed` | `declined` | `cancelled`.

❗ Fuente — M17. El cuarto estado (`declined`) fue admitido por la restricción `CHECK` de Postgres recién en la migración `20260609233130`; el hallazgo completo, con cita verbatim de ambas migraciones, se documenta en [Anexo I — Modelo de Datos](#) (Tabla 43).

Favoritos y plan destacado

El toggle de favorito actualiza el caché de TanStack Query antes de recibir respuesta del servidor (`onMutate`), para que el ícono cambie sin demora perceptible; si la mutación falla, el valor previo se restaura (`onError`) y la interfaz vuelve a su estado real. El plan destacado sigue un ciclo independiente: al activarse la suscripción, un único salón por anfitrión pasa a `is_featured = true` y gana prioridad de orden en los resultados de búsqueda por defecto; al cancelarse, ambos cambios (suscripción y bandera) se revierten en la misma operación.

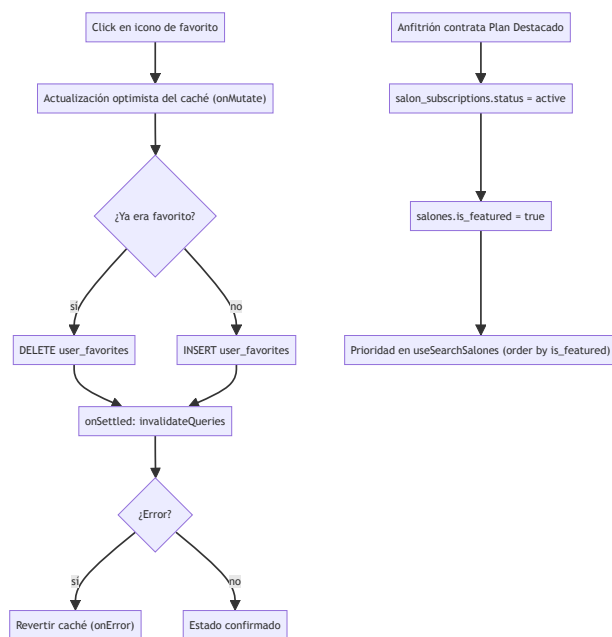


Figura 34 — Gestión de favoritos y plan destacado.

Índice de flujos

Flujo	Actor	Precondición	Resultado
Búsqueda y filtrado	Visitante	Ninguna	Lista/mapa de salones filtrados
Reserva (3 pasos)	Usuario autenticado	Sesión iniciada; salón con disponibilidad en la fecha elegida	Reserva creada en estado <code>pending</code>
Publicación de salón (4 pasos)	Anfitrión	Sesión iniciada	Salón publicado o actualizado, con imágenes en Storage
Cambio de estado de reserva	Anfitrión	Reserva en estado <code>pending</code>	Reserva en <code>confirmed</code> , <code>declined</code> (con motivo) o cotizada
Gestión de favoritos	Usuario autenticado	Sesión iniciada	Salón agregado/quitado de favoritos, con reversión ante error
Plan destacado	Anfitrión	Sesión iniciada; suscripción <code>active</code>	Salón con <code>is_featured = true</code> y prioridad en resultados

Tabla 50 — Índice de flujos: actor, precondición y resultado.

Anexo III. Backlog Completo de User Stories

Este anexo reproduce el backlog completo del proyecto: las 50 issues del repositorio (M06), con su hito de GitHub asociado y su estado real, clasificado como **entregada** (issue cerrada) o **diferida** (issue abierta al momento de esta verificación). El detalle de las 15 historias destacadas y su relación con criterios de aceptación se documenta en [Planificación Scrum](#) (Tabla 19); la ejecución cronológica, en [Ejecución por Sprint](#).

Backlog completo

Issue	Título	Hito / épica (GitHub)	Estado
#11	UI-01: High-fidelity Search UI (Filter logic, Plus Jakarta Sans)	Phase 1.B: Search & Filtering	Entregada
#12	UI-02: High-fidelity Salon Cards (Verified badge, responsive)	Phase 1.B: Search & Filtering	Entregada
#13	AUTH-01: Supabase Authentication (Login, Register, Session)	Phase 1.A: Auth & Onboarding	Entregada
#14	DATA-01: Dynamic Salon Listing: Connect to Supabase Table	Phase 1.B: Search & Filtering	Entregada
#15	SALON-01: Salon Detail Page: Gallery & Features	Phase 1.B: Search & Filtering	Entregada
#16	BOOK-01: Booking Flow: Date Selection & Availability	Phase 2: Booking & Payments	Entregada
#17	BOOK-02: Booking Confirmation & User Dashboard	Phase 2: Booking & Payments	Entregada
#18	HOST-01: Host Dashboard: Manage own listings	Phase 1.A: Auth & Onboarding	Entregada
#19	HOST-02: 'Create Salon' Wizard for Hosts	Phase 1.A: Auth & Onboarding	Entregada
#20	TEST-01: Unit & Component Testing Setup (Vitest)	Phase 2+: Polish & Optimization	Entregada
#21	TEST-02: E2E Testing for Booking Flow (Playwright)	Phase 2+: Polish & Optimization	Entregada
#22	INFRA-01: Production Infrastructure (CloudFront + SSL)	Phase 2+: Polish & Optimization	Entregada
#23	DOCS-01: Final Project Report & Handoff	Phase 2+: Polish & Optimization	Diferida
#24	PERF-01: Performance & SEO Optimization	Phase 2+: Polish & Optimization	Entregada

Issue	Título	Hito / épica (GitHub)	Estado
#25	chore(types): regenerar database.types.ts desde Supabase	Phase 2+: Polish & Optimization	Entregada
#26	fix(responsive): página no es completamente responsive en mobile	Phase 2+: Polish & Optimization	Entregada
#27	feat(salones): persistir filtros en URL o localStorage	Phase 1.B: Search & Filtering	Entregada
#28	fix(ux): agregar loading states a búsquedas y filtros	Phase 2+: Polish & Optimization	Entregada
#29	fix(ux): mejorar manejo de errores y mensajes al usuario	Phase 2+: Polish & Optimization	Entregada
#30	feat(salones): implementar paginación o infinite scroll	Phase 1.B: Search & Filtering	Entregada
#31	feat(booking): completar flujo de reserva	Phase 2: Booking & Payments	Entregada
#32	feat(backend): implementar notificaciones por email	Phase 2.B: Notifications	Entregada
#33	feat(social): implementar sistema de reviews y ratings	Phase 2+: Polish & Optimization	Diferida
#34	fix(seo): implementar meta tags y Open Graph	Phase 2+: Polish & Optimization	Entregada
#35	perf: auditar y mejorar Core Web Vitals	Phase 2+: Polish & Optimization	Diferida
#36	feat(i18n): completar traducciones español-inglés	Phase 2+: Polish & Optimization	Entregada
#37	chore(database): revisar y agregar indexes necesarios	Phase 2+: Polish & Optimization	Entregada
#38	chore(design): documentar todos los color tokens del brandbook	Phase 2+: Polish & Optimization	Diferida
#45	feat(payments): integrar Mercado Pago para reservas	Phase 2: Booking & Payments	Diferida
#46	feat(admin): panel de aprobación y moderación de salones	Phase 3.B: Admin Panel	Entregada
#47	feat(host): plan Destacado y suscripción de visibilidad para dueños	Phase 3: Host Features	Entregada
#63	feat(ui): rediseño visual v2 — Design Handoff (tokens, hero editorial, HostyBadge)	Phase 2+: Polish & Optimization	Entregada
#64	fix(ux): correcciones UX y features faltantes (Grupos 1-4)	Phase 2+: Polish & Optimization	Entregada
#65	feat(host): gestión de reservas del host (drawer, confirmar/rechazar, conflictos, RLS)	Phase 3: Host Features	Entregada

Issue	Título	Hito / épica (GitHub)	Estado
#66	feat(host): precios flexibles y catálogo de servicios extra	Phase 3: Host Features	Entregada
#67	feat(host): agenda — calendario mensual con bloqueo de fechas	Phase 3: Host Features	Entregada
#68	feat(host): gestión de salones (pausar/activar/eliminar, contacto WhatsApp/llamada)	Phase 3: Host Features	Entregada
#69	chore(db): migraciones Supabase del panel host	Phase 3: Host Features	Entregada
#70	fix(ci): commitear routeTree.gen.ts para desbloquear build de CI	Phase 2+: Polish & Optimization	Entregada
#71	fix(ci): estabilizar pipeline de CI — npm ci en frontend/, sync de lockfiles	(sin hito)	Entregada
#72	fix(nav): el link 'Cómo funciona' de la navbar no navega a ninguna sección	(sin hito)	Entregada
#73	chore(nacho): integrar cambios pendientes de Nacho	(sin hito)	Entregada
#74	fix(salones): el mapa en /salones no está implementado	(sin hito)	Entregada
#75	fix(favorites): agregar a favoritos no persiste — solo estado local	(sin hito)	Entregada
#76	fix/footer): links del footer son placeholders	(sin hito)	Entregada
#81	feat(host): geolocalizar salones al crearlos para el mapa de /salones	(sin hito)	Entregada
#85	fix: borrado de salón, horarios de reserva y validaciones del flujo	(sin hito)	Entregada
#87	fix/footer): links apuntan a rutas incorrectas o inexistentes	(sin hito)	Entregada
#88	feat(favorites): implementar persistencia de favoritos en Supabase	(sin hito)	Entregada
#92	Test cases: flujo de register/login (unit, integración y e2e)	(sin hito)	Entregada

Tabla 51 — Backlog completo de user stories con estado (entregada/diferida).

i Fuente — M06: `gh issue list --repo juanpablovaldez/hosty --state all --limit 200 --json number,title,state,labels,milestone` (verificado 2026-07-28); 45 entregadas, 5 diferidas. Ver [_\(ver documentacion-final/meta/Datos-Verificables.md\)](#).

Sobre las 5 historias diferidas

Ninguna de las cinco issues abiertas representa trabajo inconcluso dentro de su propio alcance declarado; las cuatro primeras quedaron simplemente sin cerrar al momento de esta verificación, y la quinta es este mismo informe:

- **#45** (Mercado Pago) — integración de pasarela de pago no completada dentro del período relevado.
- **#38** (tokens de color del brandbook) — tarea de documentación de diseño pendiente.

- **#35** (Core Web Vitals) — auditoría de rendimiento pendiente.
- **#33** (reviews y ratings) — con etiqueta real `post-mvp` en el propio repositorio: se trata de una decisión explícita de excluir esta funcionalidad del alcance del MVP, no de trabajo incompleto.
- **#23** (este informe final) — es la propia tarea de documentación en curso; se cierra al finalizar este cambio.

Trazabilidad: historia → issue → PR → archivo

Historia	Issue	PR	Archivo principal
Registro e inicio de sesión	#13	(sin PR con referencia explícita)	<code>frontend/src/features/auth/store/auth.store.ts</code>
Panel del anfitrión	#18	#43	<code>frontend/src/features/host/components/HostDashboardPage.tsx</code>
Asistente de publicación de salón	#19	#44	<code>frontend/src/features/host/components/SalonWizard.tsx</code>
Búsqueda y filtros	#11	(sin PR con referencia explícita)	<code>frontend/src/features/salones/components/SalonFilters.tsx</code>
Detalle de salón	#15	(sin PR con referencia explícita)	<code>frontend/src/features/salones/components/SalonDetailPage.tsx</code>
Paginación del listado	#30	#55	<code>frontend/src/features/salones/components/SalonGrid.tsx</code>
Selección de fecha y disponibilidad	#16	#54	<code>frontend/src/features/bookings/components/BookingFlow.tsx</code>
Confirmación y panel de reservas	#17	(sin PR con referencia explícita)	<code>frontend/src/features/bookings/components/MyBookingsPage.tsx</code>
Flujo de reserva completo	#31	(sin PR con referencia explícita)	<code>frontend/src/features/bookings/components/BookingFlow.tsx</code>
Gestión de reservas del anfitrión	#65	(sin PR con referencia explícita)	<code>frontend/src/features/host/components/BookingDrawer.tsx</code>
Precios y servicios flexibles	#66	(sin PR con referencia explícita)	<code>frontend/src/features/host/api/host.mutations.ts</code>
Agenda y bloqueo de fechas	#67	(sin PR con referencia explícita)	<code>frontend/src/features/host/components/CalendarView.tsx</code>
Plan destacado	#47	#80	<code>frontend/src/features/host/components/PlanCard.tsx</code>
Pruebas E2E del flujo de reserva	#21	#57	<code>frontend/src/e2e/</code>
Infraestructura de producción	#22	(sin PR con referencia explícita)	<code>infra/frontend.tf</code>

Tabla 52 — Trazabilidad historia ↔ issue ↔ PR ↔ archivo.

i Fuente — Búsqueda de referencias cruzadas issue↔PR sobre `gh pr list --state all --json number,title,body,mergedAt` (verificado 2026-07-28). Donde no se encontró una referencia textual explícita al número de issue en el título o cuerpo de ninguna PR, la celda se marca honestamente como tal en lugar de asumir una correspondencia; el archivo principal citado en esa fila sí es real y corresponde a la funcionalidad de la historia.

Anexo IV. API y Repositorio

Hosty no expone una API propia documentada con Swagger o una colección de Postman: toda la capa de datos se sirve a través de **PostgREST**, el componente de Supabase que autogenera una API REST directamente a partir del esquema de Postgres (ver [Arquitectura](#)). El contrato de esa API es el propio esquema de la base de datos, versionado como código en `supabase/migrations/*.sql`, y su proyección tipada del lado del cliente es `frontend/src/shared/lib/database.types.ts`, generado con `supabase gen types typescript`.

Operaciones PostgREST por módulo

PostgREST expone, por cada tabla del esquema `public`, operaciones `GET` (con filtros como `.eq()`, `.ilike()`, `.overlaps()`, `.range()`), `POST` (insert), `PATCH` (update) y `DELETE`, además de proyección de relaciones anidadas vía claves foráneas. En lugar de listar el máximo teórico por tabla, se cuentan las invocaciones reales de esas operaciones en el código del frontend, módulo por módulo:

Módulo	Invocaciones select	Invocaciones insert	Invocaciones update	Invocaciones delete	Total
<code>bookings</code>	3	1	1	0	5
<code>favorites</code>	2	1	0	2	5
<code>host</code>	8	4	6	3	21
<code>salones</code>	4	0	0	0	4
Total	17	6	7	5	35

Tabla 53 — Operaciones PostgREST por módulo.

i Fuente — conteo propio con `grep -oE '\.(select|insert|update|delete|upsert|rpc)\(' frontend/src/features/<módulo>/api/*.ts` (2026-07-28). Los módulos `auth`, `home`, `profile` y `errors` no tienen carpeta `api/`: `auth` opera contra Supabase Auth (GoTrue), una API separada de PostgREST, y los otros tres no consultan tablas propias.

✓ **PLACEHOLDER P-39** — URL pública del documento OpenAPI de PostgREST PostgREST publica su propio documento OpenAPI en `{SUPABASE_URL}/rest/v1/` (encabezado `Accept: application/openapi+json`). Completar `{SUPABASE_URL}` con el valor real del proyecto de Supabase de producción antes de la entrega — no se expone aquí por no ser un dato público de este repositorio. Responsable: equipo. Destino: esta sección.

✓ **PLACEHOLDER P-40** — Colección Postman curada (si la cátedra la exige) Este informe documenta el contrato autogenerado de PostgREST como equivalente funcional de Swagger/Postman. Si la evaluación requiere una colección Postman curada manualmente, confeccionarla como seguimiento posterior a esta entrega. Responsable: equipo.

Repositorio

Campo	Valor
Repositorio	<code>https://github.com/juanpablovaldez/hosty</code>
Rama principal de integración	<code>dev</code>
Ramas de entorno	<code>main</code> , <code>staging</code> , <code>dev</code>
Convención de commits	Conventional Commits, forzada por <code>commitlint</code> (<code>@commitlint/config-conventional</code>) vía hook <code>commit-msg</code> de Husky
Hook de pre-commit	<code>npx lint-staged</code> (lint sobre <code>frontend/src/**/*.{ts,tsx}</code> antes de cada commit)
Estructura	Monorepo con <code>workspaces</code> de npm (<code>frontend</code> y <code>backend</code>); <code>backend</code> sigue declarado en <code>package.json</code> pese a haber sido eliminado del disco — ver la sección 15 (Conclusiones, Tabla 40)

Tabla 54 — Estructura del repositorio y convenciones de commits y ramas.

¡ Fuente — `git remote -v` ; `git branch -a` (2026-07-28): ramas locales `main` , `dev` , `staging` , además de ramas de `features/fixes`; `commitlint.config.js` y `.husky/commit-msg` .

Workflows de CI/CD

Workflow	Disparador	Jobs	Resultado
<code>frontend-tests.yml</code>	<code>pull_request</code> sobre paths <code>frontend/**</code>	Instala dependencias con <code>pnpm</code> y ejecuta <code>pnpm test run</code> (Vitest)	Bloquea el merge si algún test falla
<code>web-dev.yml</code>	<code>push</code> a <code>dev</code> sobre paths <code>frontend/**</code> ; también <code>workflow_dispatch</code>	Build (<code>npm run build</code>), <code>aws s3 sync</code> al bucket de DEV, invalidación de CloudFront	Despliega el frontend a DEV (único ambiente desplegado, ver Testing y Calidad)
<code>infra-ci.yml</code>	<code>workflow_dispatch</code> (manual)	<code>terraform fmt -check</code> , <code>terraform init</code> , <code>terraform validate</code> , <code>terraform plan</code> sobre <code>infra/</code>	Valida cambios de infraestructura sin aplicarlos automáticamente

Tabla 55 — Workflows de CI/CD: disparador, jobs y resultado.

i Fuente — M14: `ls .github/workflows` (2026-07-28); lectura directa de `frontend-tests.yml` , `web-dev.yml` , `infra-ci.yml` .

Anexo V. Evidencias de QA

Este anexo reúne la evidencia de ejecución de la suite automatizada y el registro de defectos reales del proyecto, complementando la matriz de casos manuales de [Testing y Calidad](#).

Suite de pruebas automatizadas

Archivo	Tipo	Casos
src/features/salones/lib/pricing.test.ts	Unitaria	9
src/test/button.test.tsx	Componente	3
src/test/badge.test.tsx	Componente	3
src/features/bookings/api/bookings.test.ts	Integración	6
src/features/salones/api/salones.queries.test.ts	Integración	8
src/features/favorites/api/favorites.test.ts	Integración	6
src/features/salones/components/CardSalon.test.tsx	Componente	9
src/components/layout/Header.test.tsx	Componente	5
src/features/auth/components/LoginPage.test.tsx	Integración	1
src/features/auth/lib/auth.test.ts	Unitaria	6
src/features/auth/components/RegisterPage.test.tsx	Integración	2
src/features/auth/store/auth.store.test.ts	Unitaria	4
src/test/mocha/search-validation.test.ts	Legacy (Mocha + Chai, también recolectado por Vitest)	4
Total (Vitest)		66

Tabla 56 — Suite de pruebas automatizadas: archivo y casos.

i Fuente — `npx vitest run --reporter=verbose` ejecutado sobre el repositorio (2026-07-28): 13 archivos, 66 casos, todos en verde (Test Files 13 passed , Tests 66 passed). El conteo de casos por archivo se obtuvo con `grep -cE '^s*(it|test)\('` <archivo> sobre cada uno.

Escenarios de prueba E2E (Playwright)

Spec	Escenarios	Navegadores
<code>src/e2e/auth-flow.spec.ts</code>	9	Chromium, Firefox, WebKit
<code>src/e2e/home.spec.ts</code>	7	Chromium, Firefox, WebKit
<code>src/e2e/navigation.spec.ts</code>	8	Chromium, Firefox, WebKit
<code>src/e2e/salon-detail.spec.ts</code>	5	Chromium, Firefox, WebKit
<code>src/e2e/salones.spec.ts</code>	8	Chromium, Firefox, WebKit
Total	37 escenarios × 3 navegadores = 111 ejecuciones	

Tabla 57 — Escenarios de prueba E2E (Playwright).

❗ Fuente — `grep -cE '^s*test\(' <spec>` sobre cada archivo de `frontend/src/e2e/` (2026-07-28); configuración de navegadores en `frontend/playwright.config.ts` (`projects: chromium, firefox, webkit`).

✓ PLACEHOLDER P-44 — Anexar el reporte HTML de Playwright ya generado El proyecto ya cuenta con un reporte HTML real generado localmente en `frontend/playwright-report/index.html` (confirmado presente en el árbol de trabajo al momento de esta verificación). Anexarlo (o una captura de su resumen) como evidencia formal de la última corrida E2E antes de la entrega. Responsable: equipo. Destino: esta sección.

Evidencia de pruebas sobre la API PostgREST

✓ PLACEHOLDER P-41 — Captura de una ejecución de prueba contra la API PostgREST Adjuntar una captura de una llamada real (por ejemplo, desde el *Network tab* del navegador o desde una petición `curl` /Postman manual) contra `{SUPABASE_URL}/rest/v1/salones`, mostrando la respuesta de PostgREST. Guardar como `assets/f35-evidencia-api-postgrest.png` (nombre ya reservado en `assets/README.md`). Responsable: equipo.

Figura 35 — Evidencia de pruebas sobre la API PostgREST.

Registro de defectos y retesting

Trece incidencias reales, todas etiquetadas `bug` en GitHub y todas cerradas, constituyen el registro verificable de defectos del proyecto. La columna "Severidad" no proviene de un campo formal de GitHub (el repositorio no usa un esquema de severidad estructurado) y se marca como estimación.

Issue	Título	Severidad (estimada)	Estado	Retesting
#87	<code>fix(footer)</code> : links apuntan a rutas incorrectas o inexistentes	Baja	Cerrado	Manual, sobre DEV
#85	<code>fix</code> : borrado de salón, horarios de reserva y validaciones del flujo	Media	Cerrado	Manual, sobre DEV
#76	<code>fix(footer)</code> : links del footer son placeholders	Baja	Cerrado	Manual, sobre DEV
#75	<code>fix(favorites)</code> : agregar a favoritos no persiste (sólo estado local)	Alta	Cerrado	Manual, sobre DEV
#74	<code>fix(salones)</code> : el mapa en <code>/salones</code> no está implementado	Media	Cerrado	Manual, sobre DEV
#72	<code>fix(nav)</code> : el link "Cómo funciona" no navega a ninguna sección	Baja	Cerrado	Manual, sobre DEV
#71	<code>fix(ci)</code> : estabilizar pipeline de CI	Media	Cerrado	Verificado en <code>frontend-tests.yml</code>
#70	<code>fix(ci)</code> : commitear <code>routeTree.gen.ts</code> para desbloquear build de CI	Alta	Cerrado	Verificado en CI
#64	<code>fix(ux)</code> : correcciones UX y features faltantes (grupos 1-4)	Media	Cerrado	Manual, sobre DEV
#34	<code>fix(seo)</code> : implementar meta tags y Open Graph	Baja	Cerrado	Manual, sobre DEV
#29	<code>fix(ux)</code> : mejorar manejo de errores y mensajes al usuario	Media	Cerrado	Manual, sobre DEV
#28	<code>fix(ux)</code> : agregar <i>loading states</i> a búsquedas y filtros	Baja	Cerrado	Manual, sobre DEV
#26	<code>fix(responsive)</code> : página no es completamente responsive en mobile	Media	Cerrado	Manual, sobre DEV

Tabla 58 — Registro de defectos y retesting.

i Fuente — `gh issue list --state all --label bug --json number,title,state` (2026-07-28): 13 issues, 13 CLOSED .

⚠ **Dato simulado SIM-37** — Columna "Severidad (estimada)" GitHub no registra un campo de severidad estructurado para estos issues. La clasificación Alta/Media/Baja de esta tabla es una estimación plausible del agente, basada en el impacto funcional descrito en el título de cada issue (por ejemplo, que favoritos no persista se estima Alta por afectar datos del usuario; un link roto en el footer se estima Baja), y no corresponde a un criterio de triage formalmente documentado por el equipo.

Un defecto adicional, real y verificado —no simulado— se documenta aparte por su relevancia arquitectónica, y se distingue visualmente de la tabla anterior por no llevar el marcador `[!warning]` :

! **Fuente** — Defecto real: deriva del estado de `bookings` (M17). El estado `declined` fue utilizado por la aplicación (`frontend/src/features/host/lib/booking-status.ts`) antes de que la restricción `CHECK` de la base de datos lo permitiera (`supabase/migrations/20240101000000_init_hosty.sql:57-58` , que sólo aceptaba `pending` , `confirmed` y `cancelled`). Severidad: media. Resolución: migración `supabase/migrations/20260609233130_host_booking_management.sql:7-11` , que elimina y recrea la restricción con los 4 valores. Retest: se verificó, leyendo la migración, que la restricción `bookings_status_check` recreada admite explícitamente `'pending'` , `'confirmed'` , `'declined'` , `'cancelled'` . Análisis de deuda técnica asociado en [Testing y Calidad](#) y en la sección 15 (Conclusiones, Tabla 40).

Checklist de evidencias y capturas pendientes

✓ **PLACEHOLDER P-42** — Reporte de cobertura de pruebas No existe una herramienta de cobertura de líneas configurada en el proyecto (ver [Testing y Calidad](#), Tabla 32); por lo tanto, no hay un reporte que capturar todavía. Esta figura queda pendiente hasta que se instale una herramienta como `@vitest/coverage-v8` y se ejecute con esa opción habilitada. Responsable: equipo. Destino: esta figura y Tabla 32.

Figura 36 — Reporte de cobertura de pruebas.

✓ **PLACEHOLDER P-43** — Capturas del flujo de reserva en ejecución Adjuntar capturas de pantalla de los 3 pasos del *wizard* de reserva (`BookingFlow.tsx`) sobre el ambiente DEV desplegado, mostrando datos reales. Guardar como `assets/f37-flujo-reserva.png` (nombre ya reservado en `assets/README.md`). Responsable: equipo.

Figura 37 — Capturas de la aplicación en ejecución (flujo de reserva).

Evidencia	Estado
Reporte HTML de Playwright (<code>frontend/playwright-report/</code>)	Generado localmente; pendiente de anexo formal (P-44)

Evidencia	Estado
Corrida de Vitest reproducida en este cambio	Ejecutada: <code>npx vitest run</code> → 13 archivos, 66 casos, todos en verde (2026-07-28)
Corrida de <code>tsc -b --noEmit</code> reproducida en este cambio	Ejecutada: sin errores (2026-07-28)
Corrida de <code>eslint .</code> reproducida en este cambio	Ejecutada: 6 errores, 4 advertencias (2026-07-28; ver Testing y Calidad , Tabla 34)
Captura de la API PostgREST	Pendiente (P-41)
Reporte de cobertura de líneas	No existe herramienta configurada; pendiente de adopción (P-42)
Capturas del flujo de reserva en ejecución	Pendiente (P-43)

Tabla 59 — Checklist de evidencias y capturas pendientes.